SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECHNOLOGY, SURAT
Department of Computer Science and Engineering

Unverified Assumptions in Embedded Safety Firmware

Configuration, Specification and Response-Time Defects in an MQTT Alarm System

Research and Implementation Report

Submitted by MD Humayun
Admission No. P25IS016
Programme M.Tech, Computer Science and Engineering
Specialisation: Information Security and Privacy
Semester M.Tech II, Semester III
Academic year 2026–2027

Supervisor Dr. M. A. Zaveri
Department of Computer Science and Engineering
SVNIT Surat

Repository `github.com/mdhumayun7/Smart-Home-Automation`
Commit `5fbe1235a6633c6cef58a8e5e4c2a2c77c6b2ce4`
Report date 7 November 2026

This report accompanies a research manuscript prepared for journal submission. The manuscript and this report are separate documents with different purposes; Section 1.4 explains the relationship.

Declaration

I declare that the work presented in this report is my own, carried out under the supervision of Dr. M. A. Zaveri in the Department of Computer Science and Engineering at Sardar Vallabhbhai National Institute of Technology, Surat.

Specifically:

- The firmware, formal specifications, experimental harnesses, analysis scripts and figure generators described here were written by me and are committed to the repository named on the title page.
- Every experimental result reported is produced by those committed scripts and regenerates from the seeds recorded with it. Where a figure or table reports a number, that number is recomputed from the raw per-trial records by `experiments/validate_figures.py`.
- Work by others is cited where it is used. The only numerical results in this report that are not my own are the cycle-loss measurements of Behnke et al., which are attributed to them wherever they appear.
- **No physical hardware measurement is reported.** Chapter 12 states what has and has not been measured, and the hardware conditions described in Appendix I have not been executed.
- An AI assistant was used for drafting, documentation, code review and literature discovery. All research claims, experimental designs, results and conclusions are my own; I reviewed and verified the content and take full responsibility for it. The accompanying manuscript carries the corresponding declaration in the form the publisher requires.

MD Humayun (P25IS016) Date

This page is a draft prepared for the author's signature. It is unsigned and carries no approval.

Certificate

This is to certify that the work contained in this report, entitled *Unverified Assumptions in Embedded Safety Firmware: Configuration, Specification and Response-Time Defects in an MQTT Alarm System*, submitted by **MD Humayun** (Admission No. P25IS016) to the Department of Computer Science and Engineering, Sardar Vallabhbhai National Institute of Technology, Surat, was carried out under my supervision.

Dr. M. A. Zaveri
Supervisor
Computer Science and Engineering
SVNIT Surat

Head of Department
Computer Science and Engineering
SVNIT Surat

Template only. This page is an unsigned template reproducing the form the department normally requires. No signature, endorsement, approval or examiner comment has been obtained or reproduced, and none should be inferred from its presence here. It is included so the report can be completed through the department's own process.

Acknowledgement

I am grateful to my supervisor, Dr. M. A. Zaveri, for guidance throughout this work, and in particular for the insistence on separating what has been measured from what has been argued. Several of the corrections recorded in this report—the vacuous invariant in Chapter 9 above all—came from taking that distinction seriously rather than from any new technique.

I thank the Department of Computer Science and Engineering at SVNIT Surat for the facilities and the time that made this work possible.

The work builds directly on the measurements of Behnke et al., whose study of real-time degradation on the ESP32 under network load defined the question this project set out to answer. Their honesty about the limits of their own mitigations is what made the gap visible.

MD Humayun
November 2026

Draft. Names of anyone else who should be thanked are to be added by the author before submission.

Abstract

Domestic intrusion alarms are now built almost entirely on single-chip Wi-Fi systems-on-chip. On the ESP32, the radio driver, the TCP/IP stack, the MQTT client and the alarm application share one die, one scheduler and one interrupt controller. Two pieces of standard engineering advice are used to contain the resulting risk: pin the safety task to its own core, and gate risky features behind build-time configuration. Both are assertions about the system, and in practice neither is checked.

This work treats an ESP32 intrusion alarm as a case study and sets out to falsify both assertions rather than to demonstrate them. Three defects were found, each residing in a different kind of assumption, and each exposed by a method that costs considerably less than the system it examines. A compile-time guard accepted the combination of TLS and a broker login as evidence of a publisher's authority; under a single leaked credential the alarm fired in only 16.7% of 300 trials, against 100% for the control. A response-time decomposition exposed an unbounded command drain in the safety loop that 9,600 adversarial trials, 3,000,000 fuzzing iterations and exhaustive model checking had all passed over. The project's own temporal-logic invariant forbidding unauthenticated remote disarm proved to be vacuous in the specification that had been shipped and checked in continuous integration.

A twelve-configuration ablation over 3,600 trials shows that availability, integrity and the safety function fail independently of one another: a token-bucket rate limiter costs 41% of legitimate operator commands while contributing nothing to integrity. Broker access control and application-layer message authentication were found to fail under different compromises, a result reproduced against a live Mosquitto broker across eight scenarios. An adversary in possession of a compromised controller host—holding both a credential the access control list must permit and the command key—defeats the layered configuration and locks the legitimate owner out of all 1,250 attempted commands.

An attempt to generalise the configuration finding across 135 commit-pinned open-source embedded MQTT projects returned no hits for the guard pattern. That negative result is reported as such, and no claim is made about the prevalence of the pattern in the wider ecosystem.

The results establish *logical* isolation only. Physical timing invariance is a property of the silicon and of a closed-source radio driver; it has not been measured, no latency figure is reported, and the hardware protocol that would settle the question is documented rather than executed.

Keywords: embedded systems security; MQTT; configuration defects; specification mutation; response-time analysis; formal verification; ESP32

Contents

Declaration i

Certificate ii

Acknowledgement iii

Abstract iv

Table of Contents v

List of Figures x

List of Tables xi

List of Abbreviations xii

1 Introduction 1
- 1.1 Research background . . . 1
  - 1.1.1 Embedded safety firmware . . . 1
  - 1.1.2 MQTT-based communication . . . 2
  - 1.1.3 Security challenges in connected IoT systems . . . 2
  - 1.1.4 Why reliable intrusion alarms matter . . . 2
- 1.2 Motivation . . . 2
- 1.3 Research objectives . . . 3
- 1.4 Scope of the work . . . 4
- 1.5 Main contributions . . . 4
- 1.6 Organisation of this report . . . 5

2 Literature Review and Research Gap 6
- 2.1 Real-time degradation from on-chip network activity . . . 6
- 2.2 Timing isolation on larger platforms . . . 6
- 2.3 Radio-layer attacks on this device class . . . 7
- 2.4 Finding defects in embedded firmware . . . 7
- 2.5 Security and verification on the ESP32 application stack . . . 8
- 2.6 Research gap . . . 8
- 2.7 How this work differs . . . 9

3 System Requirements and Threat Model 10
- 3.1 Functional requirements . . . 10
- 3.2 Non-functional requirements . . . 10
- 3.3 System model . . . 11
  - 3.3.1 Two claims requiring different evidence . . . 11
- 3.4 Security objectives . . . 12
- 3.5 Threat model . . . 12
  - 3.5.1 The primary adversary . . . 12
  - 3.5.2 Extended compromise scenarios . . . 12
  - 3.5.3 E and E′: why the distinction decides the result . . . 12
  - 3.5.4 Trust boundaries . . . 13

3.5.5 Authentication and authorisation assumptions . . . 13
3.5.6 Limitations of the threat model . . . 13

4 Proposed System Architecture 15
4.1 Overview . . . 15
4.2 Hardware platform . . . 15
4.3 Core 1: the safety domain . . . 15
4.3.1 The state machine . . . 17
4.4 Core 0: the network domain . . . 17
4.5 What the two domains share . . . 17
4.5.1 The asymmetric mutex timeout . . . 18
4.6 The command path . . . 18
4.7 Security architecture . . . 20
4.8 Operator and controller interactions . . . 20
4.9 What the architecture does not contain . . . 21

5 Technologies, Tools and Development Environment 22
5.1 Target platform . . . 22
5.1.1 Firmware dependencies . . . 22
5.2 Verification and experimentation environment . . . 22
5.2.1 Why the firmware is tested against host stubs . . . 23
5.3 Resource cost on the target . . . 24
5.4 Simulation and modelling tools . . . 24
5.5 Code quality and verification infrastructure . . . 24
5.6 What the environment could not do . . . 25

6 Detailed Implementation 26
6.1 Module map . . . 26
6.2 Constants and the build configuration . . . 26
6.3 Data structures . . . 27
6.4 Core 1: the safety loop . . . 27
6.4.1 Input conditioning . . . 27
6.4.2 The state machine . . . 27
6.4.3 Output driving . . . 27
6.5 The command path . . . 27
6.5.1 Authentication on core 0 . . . 27
6.5.2 Dispatch on core 1 . . . 28
6.6 The configuration guard . . . 29
6.6.1 The defect . . . 29
6.6.2 The repair . . . 29
6.6.3 A second guard: the published demo key . . . 30
6.7 Response-time analysis and the unbounded drain . . . 30
6.7.1 The decomposition . . . 30
6.7.2 The defect the analysis exposed . . . 31
6.7.3 Why this defect is the most instructive result in the project . . . 31
6.8 The network task . . . 32
6.9 Event reporting . . . 32
6.10 Resource cost . . . 32
6.11 How the implementation was tested . . . 32

7 Experimental Methodology 34
7.1 Evidence categories . . . 34
7.2 Hypotheses . . . 34
7.3 Determinism and reproduction . . . 35
7.4 Metrics . . . 35
7.5 The experiments . . . 35
7.5.1 E1 and E2: the adversarial campaign . . . 35
7.5.2 E3: the command-path ablation . . . 36
7.5.3 E4: host-build fuzzing . . . 36
7.5.4 E5: detector parameter sensitivity . . . 36
7.5.5 E6: access control against a live broker . . . 37
7.5.6 E7: the open-source corpus study . . . 37
7.5.7 E8 and E9: conformance and model checking . . . 37
7.6 The experimental setup as a whole . . . 37
7.7 Defects found in the measurement infrastructure . . . 37

8 Results and Statistical Analysis 39
8.1 Statistical treatment . . . 39
8.2 Trial accounting . . . 39
8.3 E1 and E2: the adversarial campaign . . . 40
8.4 E3: the command-path ablation . . . 40
8.4.1 The six command-path designs . . . 40
8.4.2 Three findings . . . 40
8.5 Broker authorisation against message authentication . . . 41
8.5.1 The nine-cell matrix . . . 41
8.5.2 Neither control dominates . . . 42
8.5.3 An unanticipated negative result: owner lockout . . . 42
8.5.4 An overclaim found by measuring the missing arm . . . 42
8.6 E6: results against a live broker . . . 42
8.7 E5: detector parameter sensitivity . . . 44
8.8 E4: fuzzing . . . 44
8.9 E7: the corpus study, and a negative result . . . 44
8.9.1 Rule 1 found nothing . . . 44
8.9.2 The rule was wrong at first . . . 45
8.9.3 Rule 2 and the nine confirmed projects . . . 45
8.10 E8 and E9: conformance and model checking . . . 45
8.11 Summary of statistical values . . . 45

9 Formal Verification 47
9.1 Why formal methods were applied at all . . . 47
9.2 Specification 1: the alarm state machine . . . 47
9.2.1 What is modelled . . . 47
9.2.2 Properties checked . . . 47
9.2.3 Result . . . 48
9.3 The vacuity defect, and the repair . . . 48
9.3.1 What was wrong . . . 48
9.3.2 Why this mattered more than an ordinary bug . . . 48
9.3.3 The repair . . . 49
9.3.4 The third configuration: a check that must fail . . . 49
9.4 Specification 2: the isolation model . . . 49
9.4.1 What is modelled . . . 49

9.4.2 Properties checked . . . 49
9.4.3 Result . . . 50
9.4.4 What this model does and does not establish . . . 50
9.5 Trace conformance: connecting the model to the code . . . 50
9.6 Summary . . . 51

10 Security Evaluation 52
10.1 The threat model, restated with outcomes . . . 52
10.1.1 Why E and E′ are the decisive pair . . . 52
10.1.2 An asymmetry in cost . . . 53
10.2 Flash extraction and what is actually protected . . . 53
10.2.1 The threat . . . 53
10.2.2 Why the device survives it anyway . . . 53
10.2.3 What it does not mean . . . 53
10.3 The compile-time guard . . . 53
10.3.1 The original defect . . . 53
10.3.2 The repair, and why it is a build-time error . . . 54
10.3.3 A second guard: the published key . . . 54
10.4 Availability as a security property . . . 54
10.5 Properties established, and by which instrument . . . 55
10.6 Properties explicitly not established . . . 55

11 Figures, Diagrams and Visual Documentation 57
11.1 Generation pipeline . . . 57
11.2 The eight manuscript figures . . . 57
11.3 Quality criteria, and how each is enforced . . . 58
11.3.1 The printed-size defect . . . 58
11.3.2 The label-fit defect . . . 58
11.3.3 The baked-in cross-reference defect . . . 59
11.3.4 Colour semantics . . . 59
11.4 Verification of figure values against raw records . . . 59
11.5 Tables . . . 59
11.6 Figure collection for submission . . . 59

12 Limitations and Threats to Validity 61
12.1 The central limitation: no hardware measurement . . . 61
12.2 Threats to internal validity . . . 62
12.2.1 The harness is not the firmware . . . 62
12.2.2 Four defects were found in the measurement infrastructure itself . . . 62
12.2.3 The vacuous invariant . . . 62
12.2.4 No multiplicity correction . . . 62
12.3 Threats to external validity . . . 63
12.3.1 A single system . . . 63
12.3.2 The corpus is biased by construction . . . 63
12.3.3 One broker, one version, one configuration . . . 63
12.3.4 One framework, two versions . . . 63
12.4 Threats to construct validity . . . 63
12.5 Scope exclusions . . . 64
12.6 Known unresolved design problems . . . 64
12.7 Items requiring confirmation by the author or supervisor . . . 64

13 Conclusion and Future Work **66**
13.1 What the work set out to do, and what it found . . . 66
13.2 Objectives against outcomes . . . 66
13.3 What this work establishes . . . 67
13.4 What this work does not establish . . . 68
13.5 Future work . . . 68
13.5.1 Immediate: execute the pre-registered hardware protocol . . . 68
13.5.2 Repair the two open design problems . . . 68
13.5.3 Extend the evaluation . . . 69
13.5.4 Generalise the method rather than the finding . . . 69
13.6 Closing remark . . . 69

References **70**

A Repository Structure **73**

B Source Code Inventory **75**

C Firmware Configuration Reference **77**
C.1 Build-time switches . . . 77
C.2 Compile-time constants . . . 77
C.3 Command wire format . . . 78

D Formal Specification Listings **79**

E Test Suite Reference **80**

F Build and Deployment Instructions **81**
F.1 Prerequisites . . . 81
F.2 Firmware . . . 81
F.3 Verification and experiments . . . 81
F.4 Documents . . . 81
F.5 Deployment guidance . . . 81

G Experiment Catalogue **82**

H Figure Index **83**

I Hardware Validation Protocol **84**
I.1 The probe that established hardware was unavailable . . . 84
I.2 The rig . . . 84
I.3 The four conditions . . . 84
I.4 The pre-registered analysis . . . 85
I.5 Status of the tooling . . . 85

J Journal Submission Status **86**

K Reproducibility Record **87**
K.1 What was re-verified during the preparation of this report . . . 87
K.2 Determinism . . . 87
K.3 What cannot be reproduced . . . 87

List of Figures

4.1 System architecture 16
4.2 Command execution flow 19
4.3 Broker access control architecture 20

6.1 Compile-guard comparison 30

7.1 Experimental setup 38

8.1 Command-path ablation results 40
8.2 Security compromise comparison 41
8.3 Transport security versus command authorisation 43

List of Tables

3.3 Adversary scenarios 12

5.4 Firmware resource cost 24

List of Abbreviations

Abbreviation	Expansion
ACL	Access Control List
ASan	AddressSanitizer
APC	Article Publishing Charge
CI	Continuous Integration
CID	Character Identifier (font encoding)
CRediT	Contributor Roles Taxonomy
DUT	Device Under Test
FSM	Finite State Machine
GPIO	General-Purpose Input/Output
HMAC	Hash-based Message Authentication Code
I2C	Inter-Integrated Circuit
IoT	Internet of Things
ISR	Interrupt Service Routine
JCR	Journal Citation Reports
JSA	Journal of Systems Architecture
LoC	Lines of Code
MAC	(a) Message Authentication Code; (b) Medium Access Control. Disambiguated at each use
MQTT	Message Queuing Telemetry Transport
NVS	Non-Volatile Storage (ESP32 key–value flash store)
OLED	Organic Light-Emitting Diode
PIR	Passive Infrared (motion sensor)
PUBACK	MQTT publish acknowledgement
QoS	Quality of Service (MQTT delivery class)
RF	Radio Frequency
RTOS	Real-Time Operating System
SCIE	Science Citation Index Expanded
SHA	Secure Hash Algorithm
SJR	SCImago Journal Rank
SNIP	Source Normalized Impact per Paper
SoC	System-on-Chip
SVG	Scalable Vector Graphics
TLA+	Temporal Logic of Actions (specification language)
TLC	The TLA+ model checker
TLS	Transport Layer Security
UBSan	UndefinedBehaviorSanitizer
WCET	Worst-Case Execution Time

Chapter 1

Introduction

1.1 Research background

A domestic intrusion alarm has one obligation. When a sensor detects an intrusion, a siren must sound within a bounded time. Everything else the device does — reporting state to a phone, accepting remote commands, logging events — is secondary, and none of it excuses a failure to make a noise.

The hardware on which these devices are now built makes that obligation harder to meet than it first appears. A decade ago an alarm panel was a microcontroller with a dedicated radio module or a wired connection. Today the entire device is typically one inexpensive system-on-chip. On the Espressif ESP32, which is the platform studied here, a single die carries two processor cores, the 802.11 radio and its closed-source driver, the TCP/IP stack, the TLS implementation, the MQTT client and the alarm application itself. They share the scheduler, the memory bus and the interrupt controller.

This matters because it creates a path from the outside world to the safety function that does not exist in a system with separated hardware. An attacker who cannot reach the alarm's logic directly may still be able to load the shared resources enough to delay it. The attacker needs no credentials, no access to the building and no knowledge of the application: radio-layer frames are processed beneath the IP stack whether or not any socket is listening.

1.1.1 Embedded safety firmware

The term *safety firmware* is used in this report for the portion of an embedded program whose correct and timely execution is the reason the device exists. In an alarm this is the sensor-conditioning code, the state machine that decides whether the system is armed, and the output driver that energises the siren.

Two properties distinguish safety firmware from the rest of a program. First, its obligation is a *bound*, not an average: an alarm that usually sounds in 150 ms and occasionally takes four seconds has failed, even though its mean is excellent. Second, it must continue to work when the rest of the system does not. A loss of network connectivity, a broker outage or a saturated radio must not prevent the siren from sounding, because the circumstances under which an alarm is most needed are precisely the circumstances in which the network is least reliable.

Developers of such firmware on dual-core microcontrollers are given two standard pieces of advice, which this work takes as its subject:

1. **Pin the safety task to its own core**, and place everything network-facing on the other, so that network work cannot delay safety work.
2. **Gate risky features behind build-time configuration**, so that a dangerous capability is compiled out unless the build deliberately asks for it.

Both are reasonable. Both are also *assertions* about the system rather than demonstrated properties — the first asserts something about timing, the second something about security — and in ordinary practice neither is tested. That gap is the subject of this work.

1.1.2 MQTT-based communication

MQTT is the dominant application protocol for this class of device. It is a publish–subscribe protocol in which clients connect to a central broker, publish messages to topics, and subscribe to topics they wish to receive. Its appeal for constrained devices is real: the protocol is small, the client libraries are small, and a device behind a domestic router needs no inbound connectivity, because it establishes an outbound connection to the broker and receives commands over it.

The architectural consequence is less often discussed, and it is central to Chapter 10. The broker is not a transparent pipe. It terminates each client's transport connection and *re-originates* every message it forwards. A device receiving a command therefore receives it from the broker, over the device's own TLS connection to the broker, with no cryptographic evidence of which client originally published it. Transport security protects each hop; it conveys nothing about provenance across the relay. MQTT 3.1.1 compounds this by providing no return code meaning "refused by policy": a broker that discards a publication for violating an access control list still acknowledges it, so the publisher cannot distinguish success from silent refusal.

1.1.3 Security challenges in connected IoT systems

Three difficulties recur in this device class and all three appear in the results reported here.

Credentials are shared with the device. A device that authenticates to a broker must hold a credential, and that credential lives in flash on a physically accessible object. The same is true of any application-layer key. If both are in the same image, an adversary who extracts one extracts the other — a point that turns out to determine which of two defences survives which compromise (Section 10.2).

The controlling host is part of the trust boundary. A phone or home-automation server that legitimately issues commands must hold whatever authority is needed to issue them. Its compromise therefore yields that authority, and no amount of care inside the device can prevent it. Chapter 10 reports this as the configuration that defeats the strongest design evaluated here.

Availability and integrity are separate properties and trade against each other. A rate limiter improves resistance to flooding and degrades legitimate availability. A static shared token preserves availability and provides no replay resistance. The ablation in Section 8.4 measures this independence rather than assuming it.

1.1.4 Why reliable intrusion alarms matter

The case for taking this seriously does not rest on sophistication. These devices are cheap, numerous, installed by non-specialists and rarely updated. A homeowner who installs one has made a decision about physical security on the strength of an implicit promise about timing and authority. An alarm that can be silenced from outside the building by an adversary with a leaked credential, or delayed past the point of usefulness by radio-layer noise, has broken that promise in a way that is invisible until it matters.

The installed base is also large enough that a defect in the standard engineering advice is worth more attention than a defect in any single product.

1.2 Motivation

The direct motivation is a measurement by Behnke et al. [1], who instrumented a periodic critical task on an ESP32 and subjected the device to IP packet load. They found measurable

degradation: the task lost between 2.15% and 3.8% of its cycles for packets addressed to a port it was listening on, and between 3.17% and 6.67% for an unused port, with latency rising as the microcontroller saturated. Their conclusion was that command-and-control over IP is unsuitable for hard real-time work on devices of this kind.

That result settles one question and opens three, and those three define this project.

1. **It measures the phenomenon but does not evaluate a mitigation.** The follow-up work from the same group proposes two [2, 3], but neither is available to a developer shipping a stock ESP32: the first was evaluated against a simulated network interface and never built, and the second was prototyped on a Xilinx Zynq-7000, a considerably larger platform. Notably, [3] uses the ESP32's second core to separate the observed process from the test harness rather than as a mitigation, so *core pinning is nowhere evaluated as a defence.*

2. **All the load in that line of work is applied at the IP layer.** The attack that actually threatens a wireless alarm is deauthentication — 802.11 management frames handled in the MAC layer and the radio driver, beneath the IP stack, which the device must process whether or not a socket is listening. Whether this interference path behaves like IP load is unknown.

3. **No treatment of the isolation argument is formal, and none models an adversary** rather than volumetric stress.

A second motivation emerged during the work itself and ultimately shaped the paper more than the first. In the course of trying to verify the two standard assertions, three defects were found *in this project's own artefacts* — not in third-party code. Each had survived the techniques one would normally rely on. That experience, rather than any single finding, is what the research reports.

1.3 Research objectives

O1. Determine whether core pinning combined with bounded queue discipline is adequate as an isolation mitigation on a single-chip Wi-Fi SoC, and establish precisely which part of that question can be settled without physical measurement.

O2. Establish, formally, whether an adversary with remote command-interface access can affect the safety function's control or data flow.

O3. Determine experimentally which command-path defence preserves the safety function, operator availability and replay resistance, treating the three as separate properties rather than as one.

O4. Determine where broker-side authorisation and application-layer authentication fail, and whether either dominates the other under credential compromise.

O5. Derive a response-time bound for the safety loop and identify which interference terms can be settled analytically and which require hardware.

O6. Establish whether the configuration defect found in this system generalises to comparable open-source projects.

O7. Specify a measurement protocol capable of falsifying the timing claim that remains unresolved, and publish it whether or not it can be executed.

1.4 Scope of the work

What is inside the scope

One artefact is studied: an ESP32-WROOM-32 intrusion alarm, firmware v2.2.0, whose complete source is in the repository. The evaluation covers its command path, its safety state machine, its queue discipline, its broker authorisation and its application-layer authentication. The evidence is of five kinds, kept separate throughout: formal verification, host simulation, testbed measurement against a live broker, analysis, and a static corpus study.

What is outside the scope, and why

Physical timing measurement. No latency figure is reported anywhere in this report or in the accompanying manuscript. The simulation harness advances a logical clock, so it provides no information about timing whatever. The hardware protocol that would settle the question is specified in Appendix I and has not been run.

Attack tooling. The repository publishes no procedure or tool for generating radio-layer attacks. This is deliberate, and it limits what can be reported about the one condition that would need such tooling.

Generality. One chip, one application, two framework versions, no deployment. The corpus study in Section 8.9 was the attempt to generalise and it returned nothing; that outcome is reported as a null result and no prevalence claim is made from it.

Relationship to the accompanying manuscript

Two documents exist and they serve different purposes. The manuscript is prepared for journal submission: it is concise, confined to what is defensible before reviewers, and formatted to the target journal's requirements. This report documents the whole project — including the development environment, the implementation at module level, the experimental apparatus, and the defects found in the project's own measurement infrastructure along the way, which a journal paper has no room for. Where the two overlap, the numbers are identical, because both draw on the same committed records.

1.5 Main contributions

Eight contributions were claimed in an early draft. They were reduced to five after each was classified against the literature, because the evidence does not support eight. The reduction is recorded in `docs/CONTRIBUTIONS.md`.

The central claim is this:

> A security property asserted in build configuration, and a timing property asserted in design, are both unverified claims. Each can be falsified by a method that costs far less than the system it checks.

C1. A configuration defect that costs a safety property. A compile-time guard accepted TLS plus a broker login as evidence of publisher authority. The inference is unsound because the broker re-originates messages. Under a single leaked credential the alarm fired in 16.7% of 300 trials against 100% for the control. The guard emitted no diagnostic and survived review. (Chapter 10)

C2. A response-time decomposition that exposed an unbounded drain. The decomposition removes three of four interference terms by construction and, in doing

so, revealed that the safety loop had no static bound on its own execution time. The defect had passed 9,600 adversarial trials, 3,000,000 fuzzing iterations, exhaustive model checking and 129 host tests. (Chapter 6, Section 6.7.2)

C3. A disclosed vacuous invariant, and a regression test for vacuity. The property stating that unauthenticated remote disarm never takes effect was unfalsifiable in the specification that was shipped and checked in CI, because it assumed the refusal rather than deriving it. The model-checking runner now *requires* a deliberately defective configuration to fail. (Chapter 9)

C4. A measurement of defence independence. A twelve-configuration ablation over 3,600 trials shows availability, integrity and the safety function failing independently, and shows broker authorisation and message authentication failing under different compromises — reproduced against a live broker. (Chapter 8)

C5. A published, falsifiable measurement protocol for the timing question that remains open, together with an honest negative generalisation result from a commit-pinned corpus of 135 projects. (Appendix I, Section 8.9)

No new mechanism, attack, protocol or formalism is claimed. Ablation, response-time analysis and specification mutation are all established techniques. What the work reports is that applying them to a system's *assumptions*, rather than to its running behaviour, caught three defects that testing the running behaviour did not.

1.6 Organisation of this report

Chapter 2 reviews the relevant literature and states the research gap. Chapter 3 gives the requirements, system assumptions and threat model. Chapter 4 presents the architecture. Chapter 5 documents the development environment with verified versions. Chapter 6 explains the implementation module by module with code excerpts. Chapter 7 documents the experimental methodology and Chapter 8 the results and their statistical treatment. Chapter 9 covers formal verification. Chapter 10 discusses the security evaluation. Chapter 11 catalogues the figures. Chapter 12 states the limitations and threats to validity, and Chapter 13 concludes. Eleven appendices carry the inventories, commands and reproduction instructions.

Chapter 2

Literature Review and Research Gap

This chapter reviews the work this project builds on and states precisely where it stops. The emphasis is on what each line of work establishes and what it leaves unresolved, because the gap this project addresses is defined by the second of those more than the first.

All twenty-two references cited here are the ones carried by the accompanying manuscript. No reference has been added for this report without its bibliographic details being checked against the publisher, proceedings or preprint record.

2.1 Real-time degradation from on-chip network activity

The closest prior work, and the direct motivation for this project, is the study by Behnke et al. [1]. They instrumented a periodic critical task on an ESP32 and applied IP packet load while measuring the task's lateness. The degradation they report is modest in percentage terms but unambiguous: between 2.15% and 3.8% of cycles lost for traffic addressed to a listened port, and between 3.17% and 6.67% for an unused port, with latency rising as the microcontroller approached saturation. Their conclusion is that command-and-control over IP is unsuitable for hard real-time work on this class of device.

Two aspects of that result deserve attention. The first is that interference is higher for the *unused* port than for the listened one, which is counter-intuitive until one notices that it is consistent with cost being incurred below the application: the device pays for frame reception and demultiplexing whether or not anything is waiting for the data. The second is that the authors are explicit about the limits of their own mitigations, and it is that honesty that makes the gap visible.

The same group proposed two mitigations in follow-up work. The first is a priority-aware multiqueue network interface with per-flow interrupt moderation [2]; the second is driver-level early demultiplexing [3]. Neither is available to a developer shipping a stock ESP32 today. The multiqueue design was evaluated against a *simulated* network interface and was never built. The demultiplexing work was prototyped on a Xilinx Zynq-7000 — a platform an order of magnitude larger than an ESP32 — with measurements taken on a single core.

The detail that matters most for this project appears in [3]: the ESP32's second core is used there to separate the observed processes from the test harness, not as a mitigation. **Core pinning is therefore nowhere evaluated as a defence** in this line of work, despite being the mitigation that developers of these devices are actually told to use.

Fischer et al. [4] examine industrial device architectures under denial-of-service conditions and reach a compatible conclusion at a coarser granularity: device architecture, rather than protocol choice, determines whether real-time behaviour survives network stress. Their analysis is architectural rather than experimental at the task level, so it frames the problem without constraining the answer for any specific chip.

2.2 Timing isolation on larger platforms

A substantial body of work addresses timing isolation where the hardware offers the means to achieve it. Meszlényi et al. [5] develop timing isolation for mixed-criticality communication, and Hoornaert et al. [6] present task-based isolation for mixed-criticality workloads. Burns and Davis's review [7], now in its fourteenth edition, surveys the field as a whole.

The common prerequisite in this literature is hardware support: memory-bandwidth regulation, cache partitioning, interrupt steering, or a hypervisor. None of these exists on an ESP32. The platform offers core affinity and the scheduler's priority mechanism, and nothing else. The mixed-criticality literature is therefore highly relevant as a statement of what adequate isolation requires, and inapplicable as a source of techniques for this device class. That mismatch — a well-developed theory whose preconditions the hardware does not satisfy — is part of why the standard advice for the ESP32 remains unverified rather than merely weak.

2.3 Radio-layer attacks on this device class

The attack that threatens a wireless alarm is not IP flooding. It is deauthentication: 802.11 management frames handled in the MAC layer and the radio driver, below the IP stack, which the device must process regardless of what any application is doing.

Kremser and Graffi [8] evaluate the ESP32-S3 for Wi-Fi penetration testing and demonstrate that the hardware is itself a capable attack platform, which establishes that the capability required is cheap and widely available. Cayre et al. [9] examine the ESP32's own wireless stack offensively, finding vulnerabilities at the system-on-chip level. Mudraje et al. [11] reverse-engineer the ESP32-C3's Wi-Fi drivers, work undertaken precisely because the driver is closed source — a fact with direct consequences for this project, since an unanalysable driver is why one interference term in Chapter 6's response-time decomposition cannot be settled analytically.

Carbajal and Jodeiri Akbarfam [10] provide a software-defined testbed for quantifying deauthentication resilience. Their work is methodologically the nearest to the hardware protocol specified in Appendix I, and it reinforces a constraint observed here: experiments of this kind require an isolated radio environment, which is an institutional rather than a technical prerequisite.

What this literature establishes is that the radio-layer attack path is real, cheap to mount and distinct from the IP path. What it does not do is measure the *safety* consequence on a device whose safety task is pinned to a separate core. That is the question this project set out to answer and, as Chapter 12 states plainly, could not answer by measurement.

2.4 Finding defects in embedded firmware

Yun et al. [16] survey fuzzing for embedded systems and identify the recurring obstacle: embedded firmware is difficult to execute outside its target hardware, so fuzzing campaigns must either run on the device, at low throughput, or rehost the firmware, with fidelity loss.

Two recent systems address rehosting directly. Bley et al. [17] present protocol-aware firmware rehosting for fuzzing embedded network stacks; Tharindu et al. [18] combine system-level firmware fuzzing with selective symbolic execution. Both are substantially more capable than the approach taken here, and the comparison is worth stating honestly: the fuzzing in this project compiles the parser *for the host* and exercises it under sanitizers. That is appropriate for the question asked — whether the command parser mishandles malformed input — and it is explicitly *not* firmware rehosting. It says nothing about faults depending on the Xtensa toolchain's code generation, on the real memory map, or on the radio driver's own parsing. Chapter 12 records this as a limitation rather than leaving it implicit.

The more important point for this work, however, is one the fuzzing literature does not address. All of these techniques test a *running system*. None of them tests the *assumptions*

a system was built on. A build condition that encodes an unsound security inference will compile, run, and pass every dynamic test, because dynamic testing exercises the build that was produced rather than questioning whether that build should have been permitted. The unbounded drain reported in Section 6.7.2 makes the same point from the timing side: it survived 3,000,000 fuzzing iterations because every one of those iterations exercised a path that terminated.

2.5 Security and verification on the ESP32 application stack

Azhar et al. [12] analyse the performance and security of MQTT over TLS on ESP32-based systems, establishing the cost of the transport protection this project assumes as a baseline. Ieropoulos et al. [13] place intrusion detection on constrained devices, which is a different architectural answer to a related problem.

On the MQTT protocol itself, Zeghida et al. [20] give an overview of security issues, and Alsabbagh et al. [19] demonstrate the attack class that this project's configuration defect exposes: a rogue client with valid broker credentials raiding industrial IoT systems. This citation matters for the integrity of the present work's claims. **The attack class is published and the underlying principle is textbook**, so this project claims no new vulnerability. Its contribution is narrower: that a *compile-time guard* can encode the unsound inference, emit no diagnostic, and thereby cost a safety property.

Two 2026 papers approach the same problem from the design side. Amirkhanova et al. [21] build an end-to-end encrypted data pipeline for industrial IoT, and Yu et al. [22] propose MQTT-L, a lightweight end-to-end secure protocol using renewable hash chains. Both confirm, from the direction of protocol design, the principle this project confirms from the direction of failure analysis: end-to-end authentication is required because the broker is a relay and transport security does not survive it.

On formal methods, Krichen's survey [14] covers verification and validation for the Internet of Things, including model checking, abstract interpretation and deductive methods. Lamport [15] is the reference for the specification language used here. Neither addresses the failure mode reported in Chapter 9 — a specification that assumes the property it claims to establish — which is a known hazard in the formal-methods community but, as far as this review found, is rarely disclosed in applied papers.

2.6 Research gap

Five gaps follow from the review above, and they map onto the objectives in Section 1.4.

1. **Core pinning has never been evaluated as an isolation defence on this hardware.** It is measured as an experimental convenience in [3] and recommended universally in practice, but not tested as a mitigation.

2. **Radio-layer interference has not been related to safety outcomes.** The degradation literature applies IP load; the attack literature demonstrates radio-layer capability; nobody connects the two to a safety function's deadline.

3. **The isolation argument has not been treated formally, and no work models an adversary** rather than volumetric stress.

4. **Mixed-criticality isolation techniques presuppose hardware the ESP32 lacks**, so the gap cannot be closed by importing them.

5. **Existing defect-finding techniques test running systems, not design and configuration assumptions.** This is the gap the project's central claim addresses, and it emerged from the work rather than preceding it.

2.7 How this work differs

Aspect	**Prior work**	**This work**
Object of study	the phenomenon of degradation [1]	the *mitigation* that developers are told to use
Load applied	IP layer: UDP, MODBUS/TCP	command interface, and radio layer in the specified-but-unexecuted protocol
Adversary	volumetric stress	an explicit adversary model, scenarios A–E′
Isolation argument	informal	TLA+ specification, exhaustively checked, with trace conformance to the implementation
Isolation technique	requires bandwidth regulation, cache partitioning or a hypervisor [5, 6]	core affinity and bounded queues, which is all the hardware offers
Defect finding	fuzzing and rehosting of running firmware [17, 18]	falsification of build-time, design-time and specification assumptions
MQTT security	protocol design [21, 22]	where an existing configuration guard encodes an unsound inference
Timing claim	measured on hardware	**not measured**; protocol published instead

The last row is the one to read carefully. On the question that motivated the project, prior work has measurements and this work does not. What this work contributes on that question is a bound, a decomposition that identifies the single term requiring hardware, and a protocol capable of falsifying the claim. Chapter 12 does not soften this.

Chapter 3

System Requirements and Threat Model

3.1 Functional requirements

The requirements below are those the firmware actually implements; each is traceable to the source file and function given in Chapter 6.

ID	Requirement	Where implemented
FR1	Detect motion through a passive infrared sensor, with a warm-up period and a confirmation window to suppress spurious transitions	`readSensors()`
FR2	Detect door or window opening through a switch, debounced	`readSensors()`
FR3	Arm and disarm locally through a momentary button, debounced	`readSensors()`, `stepFsm()`
FR4	Maintain a five-state alarm state machine: disarmed, exit delay, armed, entry delay, alarm	`stepFsm()`
FR5	Drive a siren, two status indicators and a local display on state change	`driveOutputs()`
FR6	Retain the armed state across a power interruption	NVS, `persistState()`
FR7	Accept remote commands `ARM`, `ARM_AWAY`, `DISARM` and `STATUS` over MQTT	`handleCommands()`
FR8	Authenticate remote commands with a message authentication code over a monotonic counter	`authmac.h`, `handleCommands()`
FR9	Publish state and events, with a retained state topic and a periodic heartbeat	`networkTask()`
FR10	Announce unavailability through an MQTT Last Will message	`networkTask()`
FR11	Continue to perform FR1–FR6 with no network connectivity	core-1 loop, by construction

FR11 is the requirement that shapes the architecture. It is the reason the safety function cannot be permitted to depend on, wait for, or be scheduled behind any network operation.

3.2 Non-functional requirements

ID	Requirement	Status in this work
NFR1	Sensor-to-siren latency bounded by an application deadline	bound *derived* analytically (Section 6.7); **not measured**
NFR2	The safety loop never blocks indefinitely	established: zero-timeout mutex acquisition and a bounded drain, after the repair in Section 6.7.2

ID	Requirement	Status in this work
NFR3	Network load must not alter the latency distribution	*logical* isolation established by model checking; **timing isolation not measured**
NFR4	Remote commands must resist capture and replay	established: monotonic counter persisted before the command is acted on
NFR5	Legitimate operator commands must survive an attack	measured: 1,250 of 1,250 admitted under flood with authentication
NFR6	The firmware must fit the device's flash and RAM	measured at build time (Section 6.10)
NFR7	All experimental results must be reproducible from committed artefacts	established: byte-identical regeneration from fixed seeds

NFR1 and NFR3 are the two requirements this work does *not* discharge, and the distinction between them and the rest is maintained throughout the report.

3.3 System model

The device is a dual-core microcontroller with one integrated radio. Write C_s for the core hosting the safety function and C_n for the core hosting all network activity. The safety function is a periodic loop that samples sensors, advances a finite state machine and drives outputs.

Let t_d be the instant at which a physical intrusion asserts a sensor line, and t_a the instant at which the siren is driven. The quantity of interest is the sensor-to-siren latency

$$L = t_a - t_d, \tag{3.1}$$

and the safety requirement is that $L \leq L_{\max}$ for a deadline $L_{\max}$ fixed by the application.

Let Λ denote the offered load on the radio in frames per second, and F_Λ the distribution of L under that load. The isolation property one wishes to establish is

$$F_\Lambda \overset{d}{=} F_0 \quad \text{for all } \Lambda \text{ in the operating range,} \tag{3.2}$$

that is, that the latency distribution under load is indistinguishable from the unloaded control.

3.3.1 Two claims requiring different evidence

Equation (3.2) decomposes into two claims, and keeping them apart is the single most important methodological decision in this work.

Logical isolation. No control- or data-flow dependency exists by which network activity can block, delay or corrupt the safety path. This is a property of the *program*, and it is decidable by model checking and simulation.

Timing isolation. Contention for shared silicon — the memory bus, caches, the interrupt controller — does not shift F_Λ beyond the deadline margin. This is a property of the *implementation on particular hardware*, and it is decidable only by measurement.

Logical isolation is necessary but not sufficient for (3.2). This project establishes the former and specifies, but does not execute, the experiment for the latter. Every claim in this report about isolation is a claim about logical isolation unless it explicitly states otherwise.

3.4 Security objectives

SO1. An adversary without credentials must not be able to change the alarm's state.

SO2. An adversary who observes a legitimate command must not be able to replay it.

SO3. An adversary must not be able to prevent a legitimate operator command from taking effect.

SO4. An adversary must not be able to suppress the alarm's response to a physical intrusion.

SO5. A build configuration must not be able to enable an unauthenticated state change silently.

SO5 is unusual as a stated objective and it is stated because this project failed it. The defect, its measured consequence and its repair are the subject of Section 10.3.

3.5 Threat model

3.5.1 The primary adversary

The primary adversary, designated **A**, is remote and has radio-layer access only: within radio range of the device, able to transmit arbitrary 802.11 frames and to publish to the broker as an unauthenticated client. Adversary A has *no* credentials, no physical access and no prior knowledge of any secret.

Unless a claim in this report explicitly names another scenario, it is a claim against adversary A.

3.5.2 Extended compromise scenarios

Five further scenarios are evaluated separately, as extended compromise analyses. They are reported in their own right and are never used to support a claim about the design.

Table 3.3: Adversary scenarios. A is the primary threat model; B through E′ are extended compromise analyses. The *Access* column is load-bearing: E is the only scenario requiring physical access to the device, and it is the one the layered configuration survives. E′ requires none, and it is the one it does not.

	Additional capability	Access	Evaluated in
A	none (radio range only)	remote	throughout
B	one shared broker credential	remote	Ch. 10
C	controller credential	remote	Ch. 10
D	device credential	remote	Ch. 10
E	device credential + command key	**physical**	Ch. 10
E′	controller credential + command key	**remote**	Ch. 10

3.5.3 E and E′: why the distinction decides the result

Two of the five are easily conflated, and the project's central access-control result turns on the difference between them.

Scenario E is a device flash dump. It yields the device's broker credential and the command key together, because both reside in the same flash image. Obtaining it requires *physical* access to the hardware — the adversary must hold the device.

Scenario E′ is a compromised controller host — the phone, the hub or the server that legitimately issues commands. It yields a credential that the access control list *must* permit, together with the same command key, and it requires no physical access at all.

The layered configuration withstands E and falls to E′. The scenario that is cheaper for an adversary to reach is therefore the one that defeats it. This is reported in Section 10.1.1 and was found by measuring an arm that an earlier draft of the work had not run; that earlier draft claimed the layered configuration "survives every case", and the claim was wrong.

3.5.4 Trust boundaries

Four boundaries are relevant, and the architecture in Chapter 4 draws all four.

1. **Between the two cores.** The safety core trusts nothing the network core produces beyond a well-formed, authenticated command delivered through a bounded queue.
2. **Between the device and the broker.** The device trusts the broker to deliver messages and *not* to attest their origin. This is the boundary the configuration defect misread.
3. **Between the broker and publishing clients.** The broker's access control list is the only mechanism distinguishing one authenticated client from another.
4. **Between the device and its own flash.** An adversary with physical access crosses this boundary, which is what makes scenario E a physical-access scenario.

3.5.5 Authentication and authorisation assumptions

- **Transport.** TLS is assumed to protect each hop against observation and modification. It is *not* assumed to convey provenance across the broker, because the broker re-originates every message.
- **Broker authentication.** Clients present credentials from a password file. Possession of a valid credential establishes identity to the broker and nothing more.
- **Broker authorisation.** A deny-by-default access control list grants each identity specific publish and subscribe permissions per topic. An identity that must publish commands cannot be prevented from publishing commands.
- **Application-layer authentication.** A command is acted on only if its HMAC-SHA256 tag verifies under a shared secret and its counter exceeds the value persisted in non-volatile storage.
- **Key distribution is out of scope.** How the command key reaches the device and the controller is not addressed. This is a genuine limitation, not an oversight.

3.5.6 Limitations of the threat model

Four limitations should be read alongside every security claim in this report.

1. **No side-channel or fault-injection adversary is modelled.** Power analysis, electromagnetic analysis, glitching and similar techniques are entirely outside the model.

2. **The broker itself is assumed honest.** A malicious or compromised broker can drop, delay or reorder messages. Message authentication prevents it forging a valid command, but availability is not protected against it.

3. **Supply-chain compromise is not modelled.** The firmware, the toolchain and the libraries are assumed to be what they claim to be.

4. **Denial of service at the radio layer is acknowledged but unmeasured.** The one adversary capability the model takes most seriously is the one whose consequence this work could not measure. Chapter 12 does not minimise this.

Chapter 4

Proposed System Architecture

Every component described in this chapter is present in the firmware. Nothing is included because it would make the architecture look more complete, and where a component is absent that a reader might expect, the absence is stated.

4.1 Overview

The architecture rests on one decision: the safety function and all network activity are placed on separate cores and share nothing but three explicitly bounded objects. Figure 4.1 shows the whole system.

4.2 Hardware platform

The device is an ESP32-WROOM-32 development board. The relevant properties are that it is dual-core — two Xtensa LX6 cores — and that its 802.11 radio shares the die with them. A single-core variant such as the ESP32-C3 cannot host this architecture at all, which is why Appendix I insists on the dual-core part for the measurement.

Peripheral	Pin	Role
PIR motion sensor (HC-SR501)	GPIO 13	Primary intrusion detector; 10 s warm-up and a 150 ms confirmation window
Door or window switch	GPIO 14	Reed or slide switch, internal pull-up, HIGH when open
Arm/disarm button	GPIO 27	Momentary, internal pull-up, LOW when pressed, 40 ms debounce
Siren (passive piezo)	GPIO 23	Driven at 2.7 kHz
Armed indicator	GPIO 26	Green
Alarm indicator	GPIO 25	Red
OLED display (SSD1306)	GPIO 21, 22	I2C at address `0x3C`, refreshed every 250 ms

4.3 Core 1: the safety domain

Core 1 hosts the Arduino `loop()` task, which is the safety function. It performs three stages in sequence on every pass.

1. **Input conditioning.** Sensor lines are sampled and debounced. The PIR is suppressed for 10 s after power-up, because the HC-SR501 produces spurious output while stabilising, and a motion indication must persist for 150 ms before it is treated as real.

2. **State machine.** The conditioned inputs and any dequeued command advance a five-state machine. The armed state is written to non-volatile storage so that a power interruption does not disarm the system.

3. **Output driver.** The siren, the two indicators and the display are driven from the current state.

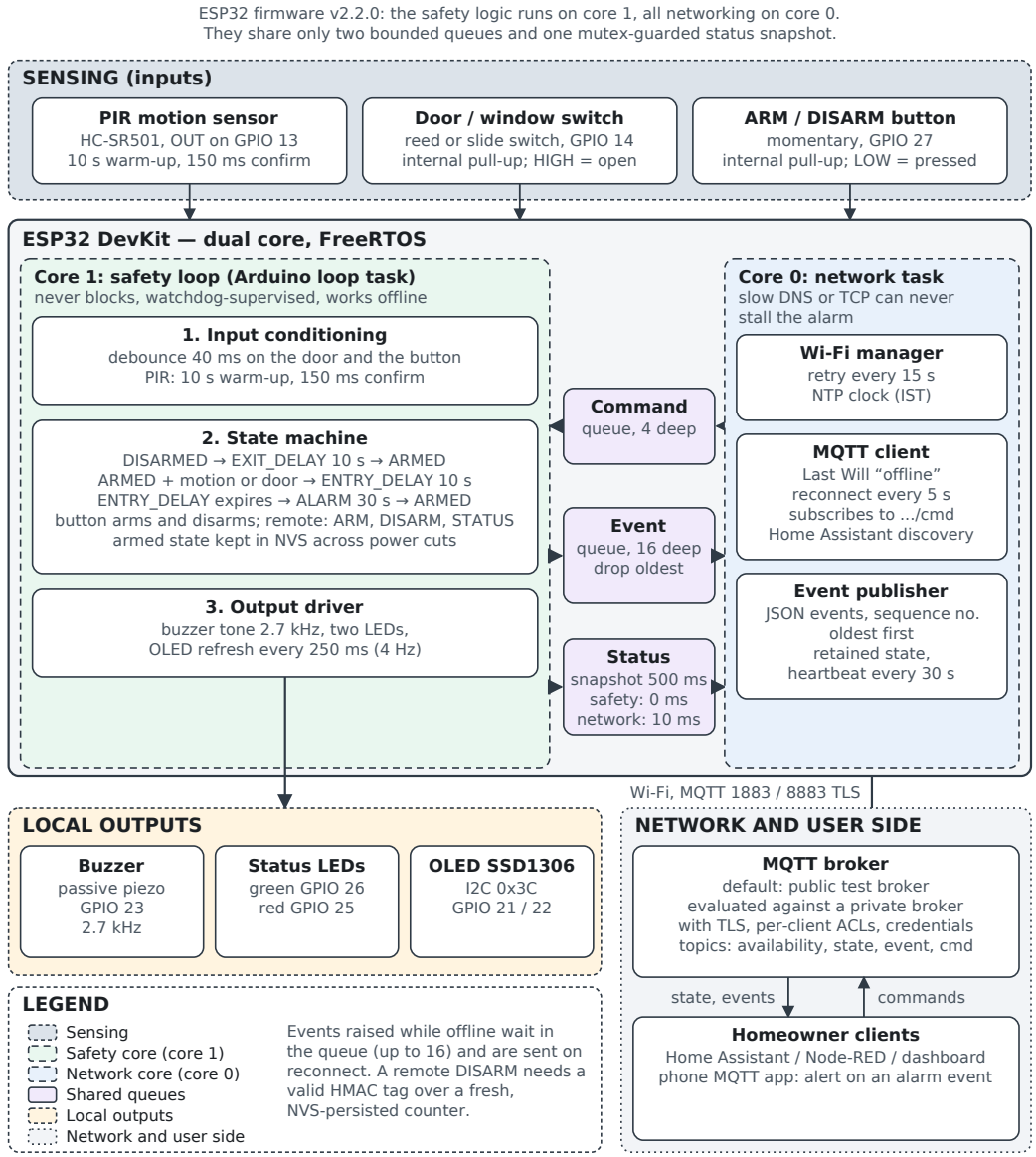


Figure 4.1: System architecture. The safety domain on core 1 (green) and the network domain on core 0 (blue) communicate only through two bounded queues and one mutex-guarded status snapshot. Every pin number, timeout and queue depth shown is transcribed from `firmware/smart_home/smart_home.ino`.

Two properties of this loop matter more than its contents. It *never blocks*: every acquisition of a shared resource uses a zero timeout and proceeds without it on failure. And it is *independent of the network*: with the radio disabled, every function in Section 3's FR1 to FR6 still works.

4.3.1 The state machine

State	Duration	Meaning and exits
`DISARMED`	—	Sensors are ignored. Exits on a local button press or an authenticated `ARM`/`ARM_AWAY`.
`EXIT_DELAY`	10 s	The occupant is leaving. Exits to `ARMED` on expiry.
`ARMED`	—	Sensors are live. Motion or a door opening moves to `ENTRY_DELAY`.
`ENTRY_DELAY`	10 s	Grace period for a legitimate entry. A disarm here returns to `DISARMED`; expiry moves to `ALARM`.
`ALARM`	30 s	The siren sounds. Returns to `ARMED` on expiry.

4.4 Core 0: the network domain

Core 0 hosts a single FreeRTOS task created explicitly rather than by the framework:

```
1 xTaskCreatePinnedToCore(networkTask, "net", 8192, nullptr, 1, nullptr, 0);
```

Listing 4.1: Network task creation, `smart_home.ino`

The final argument pins the task to core 0. The Arduino `loop()` runs on core 1 by default, so the two domains are on separate cores without any further configuration. The task performs Wi-Fi management, the MQTT client, and event publication.

- **Wi-Fi manager.** Reconnects every 15 s on loss, and maintains an NTP-corrected clock for event timestamps.
- **MQTT client.** Reconnects every 5 s, registers a Last Will message so the broker announces the device as offline, subscribes to the command topic and publishes Home Assistant discovery metadata.
- **Event publisher.** Publishes JSON events carrying a sequence number, oldest first, together with a retained state topic and a 30 s heartbeat.

Every one of these operations can block for a long time. DNS resolution, TCP connection establishment and the TLS handshake are all unbounded in the worst case. That is precisely why they are on the other core.

4.5 What the two domains share

Exactly three objects cross the boundary, and the discipline applied to each is what the isolation argument rests on.

Object	Bound	Discipline
Command queue `cmdQ`	4 entries	Core 0 enqueues non-blocking; core 1 drains at most `CMD_DRAIN_PER_PASS` entries per pass.
Event queue `eventQ`	16 entries	Core 1 enqueues non-blocking; the oldest entry is dropped when full, so the safety loop never waits for a consumer.
Status snapshot	1 struct	Guarded by `statusMutex`. Core 1 acquires with timeout **0**; core 0 acquires with a 10 ms timeout. Copied every 500 ms.

4.5.1 The asymmetric mutex timeout

The status snapshot is the mechanism most easily overlooked and it is the heart of the design. Both cores need access to the same state structure: core 1 writes it, core 0 reads it in order to publish. A conventional mutex would let the network core's acquisition delay the safety core.

The firmware avoids this by making the timeouts asymmetric:

```
/* core 1, the safety loop: never waits */
if (xSemaphoreTake(statusMutex, 0) == pdTRUE) {
    /* copy the snapshot */
    xSemaphoreGive(statusMutex);
}
/* if the lock was unavailable, the loop simply continues; the snapshot
   is refreshed on the next pass 500 ms later */

/* core 0, the network task: may wait up to 10 ms */
if (xSemaphoreTake(statusMutex, pdMS_TO_TICKS(10)) == pdTRUE) {
    /* read the snapshot and publish */
    xSemaphoreGive(statusMutex);
}
```

Listing 4.2: Asymmetric acquisition, `smart_home.ino`

The consequence is that the blocking term in the response-time analysis of Section 6.7 is *identically zero by construction* rather than merely small. The safety loop cannot be made to wait, because it never asks to. The cost is that a status publication may occasionally carry a snapshot up to 500 ms old, which is an acceptable price for a telemetry path and an unacceptable one for a safety path — which is why the asymmetry runs in this direction and not the other.

4.6 The command path

Figure 4.2 shows the full path a remote command takes, and which core performs each stage.

The ordering is deliberate. All parsing, all cryptographic verification and all rejection happen on core 0. A malformed, unsigned or incorrectly signed command is discarded there and never occupies a queue slot. Only a verified command is enqueued, so the safety core's work per command is bounded and small.

The wire format is deliberately minimal:

$$\texttt{VERB} : \texttt{CTR} : \mathrm{hex}\big(\text{HMAC-SHA256}(k,\ \texttt{"VERB:CTR"})[0{:}16]\big)$$

A verb, a counter, and a 128-bit truncated tag over both. The tag covers the counter, so an adversary cannot replay a captured command with a higher counter.

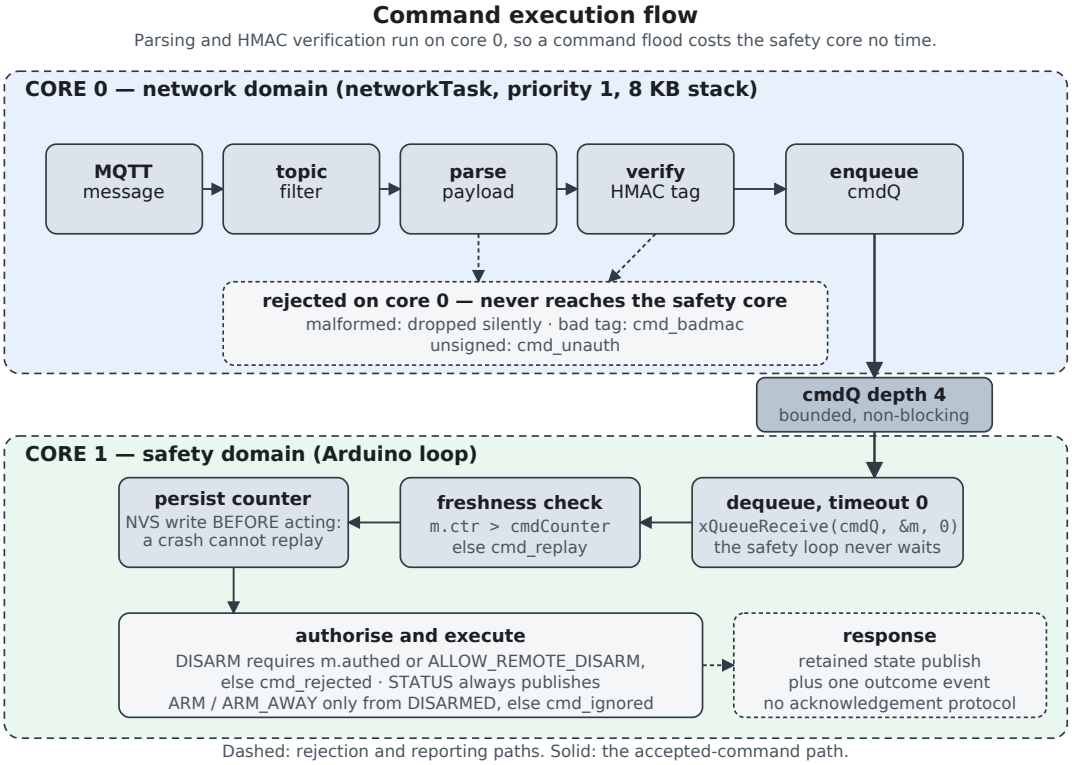


Figure 4.2: Command execution flow. Parsing and verification complete on core 0; the replay counter is committed to non-volatile storage before the command is acted on. Rejected commands never reach the safety core.

4.7 Security architecture

Three layers are available, and Chapter 10 reports what each does and does not contain. Figure 4.3 shows the broker-side permissions.

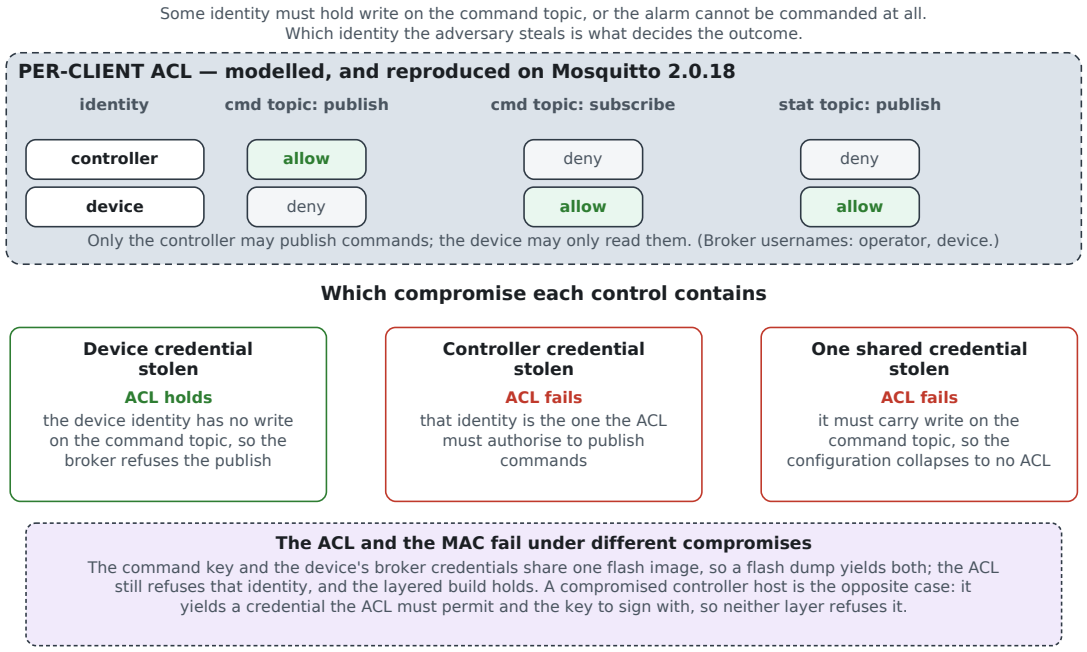


Figure 4.3: Per-client topic permissions, and which compromise each control contains. The matrix is both the configuration the ablation models and the Mosquitto `acl_file` used in the broker experiment.

1. **Transport: TLS.** Protects each hop. Does not survive the broker, because the broker terminates the connection and re-originates the message.
2. **Broker authorisation: per-client credentials and a deny-by-default access control list.** The controller identity may publish on the command topic; the device identity may only read it. This contains a compromised *device* credential and cannot contain a compromised *controller* credential, because the list must permit that identity to publish.
3. **Application layer: HMAC-SHA256 over a monotonic counter.** Survives any broker credential compromise on its own, and falls only when the key itself is compromised.

The counter is persisted to non-volatile storage *before* the command is acted on, not after. Committing afterwards would leave a window in which a power interruption between action and commit permits a replay.

4.8 Operator and controller interactions

Three parties exist in the deployed system, and their separation is what the access control list encodes.

- **The device** subscribes to the command topic and publishes state and events. In the broker configuration its username is `device`.
- **The controller** — a phone application, Home Assistant or Node-RED — publishes commands and reads state. Its username is `operator`. It is the identity the access control list must authorise to publish, which is why its compromise is scenario E′.
- **The broker** mediates. In the default configuration this is a public test broker; the evaluation used a private Mosquitto instance with TLS, per-client credentials and the access control list shown in Figure 4.3.

4.9 What the architecture does not contain

Stated explicitly, so the diagram is not read as claiming more than it shows.

- **No secure element and no hardware key storage.** The command key is in the flash image. This is why a flash dump yields it, and why scenario E exists.
- **No secure boot and no flash encryption.** Both are available on the ESP32 and neither is enabled in the artefact studied.
- **No over-the-air update path.** A deployed device would need one; its absence removes an attack surface and a capability at the same time.
- **No key rotation or provisioning protocol.** Out of scope, as Section 3 states.
- **No redundant sensing or watchdog escalation beyond the framework default.**

Chapter 5

Technologies, Tools and Development Environment

Every version in this chapter was read from the tool itself in the environment that produced the committed results. Where a version could not be verified, that is stated rather than guessed.

5.1 Target platform

Item	Version	Role and why
ESP32-WROOM-32	DevKit v1	The artefact under study. Chosen because it is the platform the research question is about: dual-core, with the radio on the same die.
Xtensa LX6 cores	2 cores	Core affinity is the only isolation primitive available; the whole architecture depends on there being two.
arduino-esp32	2.0.17 and 3.3.12	Both evaluated, because the buzzer driver API changed between them and the firmware must build under each. Resource cost is reported for both (Table 5.4).
FreeRTOS	as shipped with the above	Supplies `xTaskCreatePinnedToCore`, queues and semaphores. No version independent of the framework is pinned.

5.1.1 Firmware dependencies

Library	Purpose in this firmware
`PubSubClient`	MQTT 3.1.1 client: connection, subscription, publication, Last Will
`WiFiClientSecure`	TLS transport to the broker
`Adafruit_SSD1306`, `Adafruit_GFX`	Local OLED status display
`Preferences`	Non-volatile storage for the armed state and the replay counter
`mbedtls` (via the SDK)	HMAC-SHA256 for command authentication

5.2 Verification and experimentation environment

This is a separate environment from the target, and the distinction is material: every result reported in Chapter 8 except the broker experiment was produced here, on a host, and not on the device.

Tool	Version	Role and where it appears
GCC / g++	13.3.0	Compiles the firmware logic against host stubs, and the harnesses in `experiments/*.cpp`. Also supplies AddressSanitizer and UndefinedBehaviorSanitizer for the fuzzing campaign.
Python	3.13.18	All analysis, figure generation and verification scripts. Statistics are implemented with the standard library only, so no external package can alter a reported value.
TLA+ / TLC	TLC2 2026.10.24.013619 (rev `2ef2af8`)	Exhaustive model checking of `formal/AlarmFSM.tla` and `formal/Isolation.tla`, and of the deliberately defective configuration that must fail.
Java	OpenJDK, as installed	Runtime for `tla2tools.jar` (4,502,259 bytes, committed).
Mosquitto	2.0.18	The live broker for experiment E6. The only component in the evaluation that is external software observed rather than simulated.
`mosquitto_passwd`	with 2.0.18	Generates the password file for per-client authentication.
OpenSSL	as installed	Generates the private certificate authority and server certificate for the broker testbed.
matplotlib	3.11.2	Figures 3 and 5. Configured through `experiments/figstyle.py` with `pdf.fonttype 42`, so TrueType fonts are embedded rather than Type 3.
cairosvg	current at build	Converts the generated SVG diagrams to PDF and PNG.
Pillow	current at build	Measures text in the real font for the figure-fit check.
pdflatex	TEX Live 2024	Builds the manuscript and this report.
Pandoc	3.1.3	Produces the editable DOCX of this report.
Git	as installed	Version control; the commit hash on the title page identifies the exact state.
GitHub Actions	—	Continuous integration: re-runs the formal verification and a reduced adversarial campaign on every push.

5.2.1 Why the firmware is tested against host stubs

The firmware logic is compiled for the host against a set of stub headers in `tests/host/stubs/`, which provide the Arduino, FreeRTOS, `WiFi`, `PubSubClient`, `Preferences`, `Wire` and display interfaces.

The reason is throughput. A campaign of 9,600 adversarial trials and 3,000,000 fuzzing iterations is not feasible on the device at any useful speed, and the sanitizers that make the fuzzing meaningful do not exist for the Xtensa target. Compiling for the host makes both possible.

The cost is equally clear and is stated wherever a result from this environment is reported: **the host harness advances a logical clock**. There is no radio, no true parallel execution and no contention for shared silicon. Latency variance in this environment is identically zero, which is why no timing conclusion is drawn from it anywhere in this report.

The firmware logic itself is *not* modified for testing. The same `smart_home.ino` is compiled; only the headers it includes differ.

5.3 Resource cost on the target

Measured at build time by the Arduino toolchain, for the two framework versions. This is the one measurement in the project taken by real tooling on the real target, and it describes the binary rather than the running device.

Table 5.4: Firmware resource cost, measured at build time.

Framework version	Flash (B)	of budget	RAM (B)	of budget
arduino-esp32 2.0.17	793,433	60.5%	46,372	14.2%
arduino-esp32 3.3.12	968,221	73.9%	49,876	15.2%

The increase under the newer framework is substantial — roughly 175 kB of flash — and is a property of the framework rather than of this firmware. It is reported because a reader reproducing the build will see one figure or the other depending on which version they install.

5.4 Simulation and modelling tools

Tool	**Role**
Wokwi (`wokwi/diagram.json`)	Circuit definition for browser-based simulation of the peripherals. Used during development for wiring verification; **no reported result comes from it**.
`experiments/ablation.cpp`	The twelve-configuration harness. Models broker admission and access control in `deliverGuarded`/`admit`.
`experiments/campaign.cpp`	The adversarial campaign harness, E1 and E2.
`experiments/fuzz_cmd.cpp`	The command-parser fuzzer, run under both sanitizers.
`experiments/sensitivity.cpp`	The detector parameter sweep.

5.5 Code quality and verification infrastructure

A deliberate choice was made to treat the verification infrastructure as part of the artefact rather than as scaffolding, because four defects were found *in the harnesses themselves* during this work — two of which produced results that flattered the intended conclusion.

Check	**What it prevents**
`tests/host/run_tests.sh`	129 assertions over the firmware logic, including three negative build-guard tests that require a defective configuration to fail to compile
`formal/run_tlc.sh`	Three model-checking configurations; the third *must* report a violation, so a vacuous invariant cannot recur silently

Check	What it prevents
`experiments/test_conformance.py`	19,529 firmware transitions checked against the specification's transition relation, with a negative control
`experiments/test_harness.py`	Metamorphic and differential properties of the harness: determinism, seed sensitivity, monotone attack effort, mode separation, and agreement of the HMAC payload format with an independent implementation
`experiments/corpus/test_rule.py`	Eight controls for the static rule, including this project's own pre-repair guard
`experiments/validate_figures.py`	Recomputes every plotted value from the raw per-trial records, not from the derived summary
`experiments/check_figure_type.py`	Fails the build if any figure label would print below 6 pt
`experiments/check_figure_fit.py`	Measures every label in the real font and fails on clipping

The last two exist because a defect was found that visual inspection cannot detect: every diagram in an earlier revision printed its smallest label at roughly 4 pt, which is a function of the canvas width and the figure's scale factor rather than anything visible on screen.

5.6 What the environment could not do

Recorded here so that no reader infers a capability that did not exist.

- **No physical ESP32 hardware** was available in the environment that produced these results. There were no serial devices, no USB, no `arduino-cli` and no wireless interface. Appendix I records the probe that established this.
- **No RF-shielded enclosure**, which is a hard prerequisite for the radio-layer condition.
- **No licensed similarity-checking tool.**
- **No institutional access** to Scopus or Journal Citation Reports, which is why no journal quartile is asserted anywhere in this work.

Chapter 6

Detailed Implementation

This chapter explains the implementation at the level of individual functions and the reasoning behind them. Every excerpt is quoted from the committed source and the file and function are named, so a reader can open the original. The complete source is supplied separately as `COMPLETE_SOURCE_CODE_AND_EXPERIMENTS.zip`; the inventory is Appendix B.

Three sections carry more weight than the rest and are the ones to read if time is short: Section 6.5 on the command path, Section 6.6 on the configuration guard, and Section 6.7.2 on the unbounded drain.

6.1 Module map

File	Responsibility
`smart_home.ino`	The artefact under study. Pin definitions, timing constants, build guards, the state machine, both cores' tasks, and the command path. 713 lines of code.
`authmac.h`	HMAC-SHA256 command authentication: tag computation and constant-time comparison
`alertmsg.h`	Event record structure and serialisation
`wifi_guard.h`	Deauthentication frame counting and the detector threshold
`sensor_fusion.h`	Optional PIR and radar fusion; off in the evaluated configuration
`secrets.example.h`	Template for credentials and keys. The demo key it carries is public by design, and the build refuses to use it with authentication enabled

6.2 Constants and the build configuration

All timing and pin constants are `constexpr`, in one block, so that a reader auditing the firmware's temporal behaviour can find every value in one place.

```
constexpr uint32_t EXIT_DELAY_MS      = 10000;  // time to leave after pressing ARM
constexpr uint32_t ENTRY_DELAY_MS     = 10000;  // time to disarm after a trigger
constexpr uint32_t ALARM_DURATION_MS  = 30000;  // siren length, then system re-arms
constexpr uint32_t PIR_WARMUP_MS      = 10000;  // suits the simulator; 60000 on a real HC-SR501
constexpr uint32_t PIR_CONFIRM_MS     = 150;    // PIR must stay HIGH this long
constexpr uint32_t DEBOUNCE_MS        = 40;
constexpr uint32_t OLED_REFRESH_MS    = 250;
constexpr uint32_t STATUS_COPY_MS     = 500;    // loop -> network task snapshot rate
constexpr uint32_t HEARTBEAT_MS       = 30000;
constexpr uint32_t WIFI_RETRY_MS      = 15000;
constexpr uint32_t MQTT_RETRY_MS      = 5000;
```

Listing 6.1: Timing constants, `smart_home.ino`

`PIR_WARMUP_MS` carries a comment rather than a single value because the correct number differs between the simulator and the physical sensor. Recording that in the source is preferable to silently choosing one and misleading whoever deploys it.

6.3 Data structures

```
enum class SysState : uint8_t { DISARMED, EXIT_DELAY, ARMED, ENTRY_DELAY, ALARM };
enum class Cmd      : uint8_t { ARM, DISARM, STATUS };

struct CmdMsg {   /* one entry in the command queue */
  Cmd      cmd;
  bool     authed;   /* the MAC verified */
  uint32_t ctr;      /* the counter carried by the command */
};
```

Listing 6.2: Core data structures, `smart_home.ino`

`CmdMsg` is deliberately small and contains no pointers. It is copied by value into a FreeRTOS queue, so a command crossing the core boundary involves no shared allocation and no lifetime question. The `authed` flag is set on core 0 after verification; core 1 trusts it precisely because nothing else can write to that queue.

6.4 Core 1: the safety loop

6.4.1 Input conditioning

`readSensors()` samples the three input lines and applies two independent filters. The button and the door switch are debounced over `DEBOUNCE_MS`; the PIR is suppressed entirely for `PIR_WARMUP_MS` after boot and then required to hold its indication for `PIR_CONFIRM_MS`.

The two filters are different because the failure modes are different. A mechanical switch chatters on contact, which debouncing addresses. A pyroelectric sensor produces spurious output while its reference stabilises after power-up, and then occasional short pulses in normal operation — a warm-up suppression and a confirmation window address those.

6.4.2 The state machine

`stepFsm()` implements the five-state machine of Chapter 4. Its structure is a switch on the current state, with time-based transitions compared against a stored entry timestamp rather than counted down, so that a missed loop pass cannot lose time.

The armed state is written to non-volatile storage on every change through `persistState()`, which satisfies FR6: a power interruption while armed leaves the system armed.

6.4.3 Output driving

`driveOutputs()` is the only function that writes to the siren, the indicators or the display. Concentrating output in one function means that a reader auditing when the siren can sound has one place to look.

The buzzer driver handles an API change between arduino-esp32 2.x, which is channel-based, and 3.x, which is pin-based. The firmware supports both because both were evaluated.

6.5 The command path

6.5.1 Authentication on core 0

The wire format is a verb, a counter and a truncated tag over both:

$$\texttt{VERB} : \texttt{CTR} : \mathrm{hex}\big(\text{HMAC-SHA256}(k,\ \texttt{"VERB:CTR"})[0{:}16]\big)$$

Two design decisions are worth explaining. The tag is truncated to 128 bits (`AUTH_TAG_BYTES = 16`) because a 256-bit tag in hexadecimal would double the wire length for no practical security gain: the forgery probability against a 128-bit tag is 2^{-128}, which is far below any threshold that matters here. And the tag covers the *counter as well as the verb*, which is what prevents an adversary from taking a captured command and incrementing its counter.

Comparison uses a constant-time routine in `authmac.h`. On a device of this kind a timing side channel on tag comparison is a remote prospect, but constant-time comparison costs nothing and removes the question.

6.5.2 Dispatch on core 1

`handleCommands()` runs on the safety core and is the only consumer of the command queue.

```
static void handleCommands(uint32_t now) {
  CmdMsg m;
  for (int drained = 0;
       drained < CMD_DRAIN_PER_PASS && xQueueReceive(cmdQ, &m, 0) == pdTRUE;
       drained++) {
    // Marker from the network task for a command whose MAC did not verify.
    if (!m.authed && m.ctr == 0xFFFFFFFFu) { emitEvent("cmd_badmac", "remote"); continue; }

    if (m.authed) {
      // Reject a replay: the counter must strictly advance.
      if (m.ctr <= cmdCounter) { emitEvent("cmd_replay", "remote"); continue; }
      persistCounter(m.ctr);
    }
#if AUTH_COMMANDS
    else {
      // Authentication is required but this command was unsigned.
      emitEvent("cmd_unauth", "remote");
      continue;
    }
#endif
    const char* src = m.authed ? "remote_auth" : "remote";
    switch (m.cmd) {
      case Cmd::ARM:
        if (state == SysState::DISARMED) beginArm(src, now);
        else emitEvent("cmd_ignored", src);
        break;
      case Cmd::DISARM:
        if (m.authed || ALLOW_REMOTE_DISARM) {
          if (state != SysState::DISARMED) disarm(src, now);
          else emitEvent("cmd_ignored", src);
        } else {
          emitEvent("cmd_rejected", src);
        }
        break;
      case Cmd::STATUS:
        statusDirty = true;
        break;
    }
  }
}
```

Listing 6.3: Command dispatch, `handleCommands()` in `smart_home.ino`

Reading this block by block:

The loop header is the bounded drain discussed in Section 6.7.2. The `0` timeout on `xQueueReceive` means the safety loop never waits for a command.

The `cmd_badmac` marker is how core 0 reports a verification failure without performing any dispatch itself. The sentinel counter `0xFFFFFFFF` distinguishes it from a genuine unsigned command.

The replay check requires the counter to *strictly* advance, and `persistCounter()` commits the new value *before* the command is acted on. Committing afterwards would leave a window in which a power interruption between action and commit permits the same command to be replayed.

The `AUTH_COMMANDS` branch is where an unsigned command is refused when authentication is required. This is the clause the configuration guard of Section 6.6 is supposed to protect.

The `DISARM` case is the one that matters for security. A validly signed command proves the key holder sent it, so disarm is safe irrespective of `ALLOW_REMOTE_DISARM`. The disjunction exists for a deployment that uses TLS and a broker login without command signing — and the soundness of *that* configuration is exactly what this project found to be unjustified.

`cmd_ignored` versus `cmd_rejected` are distinguished deliberately: the first means a well-authorised command that does not apply in the current state, the second means a command that was not authorised. Conflating them would make the event log useless for exactly the analysis this project performed.

6.6 The configuration guard

6.6.1 The defect

The original guard permitted `ALLOW_REMOTE_DISARM` without `AUTH_COMMANDS` provided that TLS and a broker login were configured. The reasoning was that a deployment with transport security and broker authentication has established who may publish.

The reasoning is unsound, and it is unsound for a structural reason rather than through an oversight in the code. The broker terminates each client's TLS connection and re-originates every message it forwards, so an arriving command carries no evidence of which client published it. `MQTT_HAS_LOGIN` describes *this device's own build*, not any publisher's authority. One leaked credential — any credential permitted to publish on the command topic — is therefore sufficient to disarm the alarm, and the guard compiled such a build without complaint.

6.6.2 The repair

```
#if ALLOW_REMOTE_DISARM && !AUTH_COMMANDS && !defined(ACKNOWLEDGE_UNAUTHENTICATED_DISARM)
#  error "ALLOW_REMOTE_DISARM without AUTH_COMMANDS 1 permits anyone who can publish to the \
command topic to switch the alarm off, and TLS plus a broker login does not prevent it (see \
docs/SECURITY_ANALYSIS.md). Set AUTH_COMMANDS 1, or define \
ACKNOWLEDGE_UNAUTHENTICATED_DISARM to accept that risk deliberately."
#endif
```

Listing 6.4: The corrected guard, `smart_home.ino`

The repair removes the unsound inference rather than adding a mechanism. The transport test is gone; what remains is a requirement for either command signing or an explicit acknowledgement that the risk is being accepted. The latter exists because the research baseline must be able to build the defective configuration in order to measure it — and it must do so deliberately.

Figure 6.1 shows both versions of the decision.

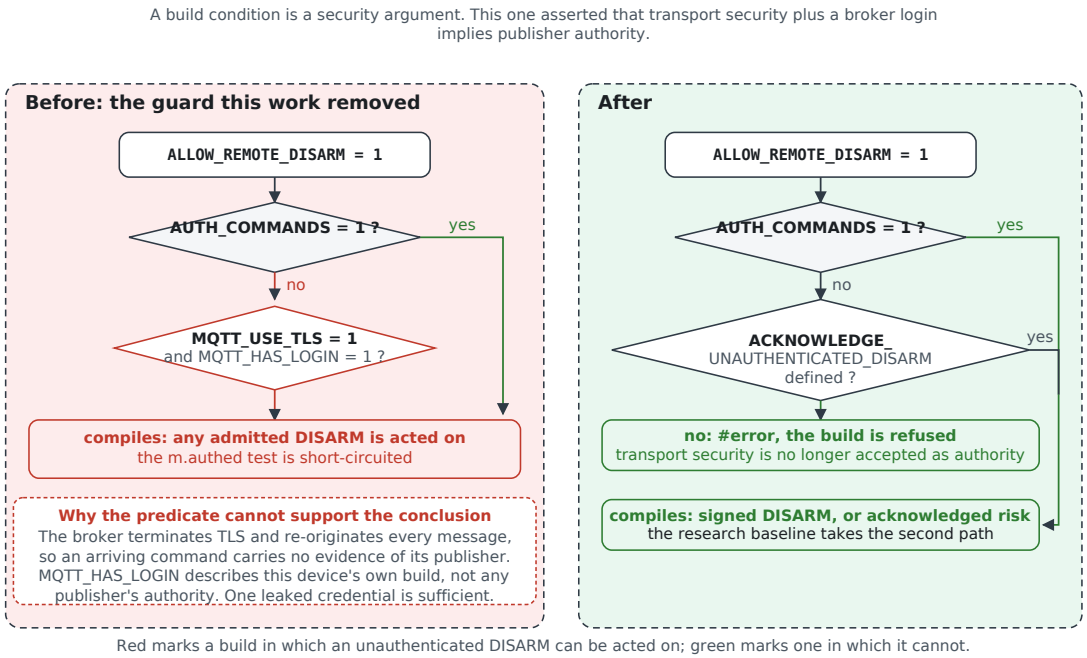


Figure 6.1: The defective guard and its repair, transcribed from the preprocessor conditions. Both branches of the original reach the same outcome, which is the defect: a signed build reaches it legitimately and an unsigned one through the unsound transport test.

6.6.3 A second guard: the published demo key

A related defect was found and closed at the same time. The repository ships a `secrets.example.h` whose demo command key is public by design. A build enabling `AUTH_COMMANDS` without replacing it would log every command as authenticated while any reader of the repository could forge one.

```
#if AUTH_COMMANDS && CMD_KEY_IS_DEMO
#  error "AUTH_COMMANDS 1 requires a real SECRET_CMD_KEY in secrets.h. The fallback key is \
published in this repository, so a command signed with it is forgeable by any reader."
#endif
```

Listing 6.5: Refusing the published key, `smart_home.ino`

Authentication that verifies against a published key is not authentication. The build now refuses rather than producing a binary that reports false assurance.

6.7 Response-time analysis and the unbounded drain

6.7.1 The decomposition

The response time of the safety loop decomposes into the loop's own execution, interference from the network core, blocking on shared resources, and residual hardware contention. Three of the four are settled by construction.

Blocking is identically zero. The safety loop acquires the status mutex with timeout `0` and proceeds without it on failure (Listing 4.2). It never waits, so no priority inversion and no blocking term exists.

Queue interference is bounded. Both queues are bounded, both enqueues are non-blocking, and the event queue drops its oldest entry when full rather than making the

producer wait.

Scheduling interference is bounded by core affinity. The network task is pinned to core 0 and the safety loop runs on core 1, so the network task cannot be scheduled ahead of the safety loop.

Residual hardware contention cannot be settled analytically. Contention for the memory bus, the caches and the interrupt controller remains, and it depends on the silicon and on a closed-source radio driver. This is the single composite term that requires measurement, and it is why Appendix I exists.

6.7.2 The defect the analysis exposed

Performing that decomposition revealed something the author had not been looking for. The command drain was an unbounded loop:

```
/* BEFORE -- no static bound on the loop's own execution time */
while (xQueueReceive(cmdQ, &m, 0) == pdTRUE) {
    /* ... dispatch ... */
}
```

Listing 6.6: The defective drain, before the repair

The safety loop therefore had no static bound on its own execution time. Core 0 can enqueue while core 1 dequeues, and in the default build — with `AUTH_COMMANDS` disabled — enqueueing costs only a parse, with no HMAC computation to throttle the producer. The loop would still terminate in practice, but only through a probabilistic argument about relative costs. That is not an acceptable basis for the one code path whose timing the entire design exists to protect.

```
// Capacity of the command queue, and the bound on how many are drained per loop pass.
static constexpr int CMD_Q_LEN = 4;

// The bound equals the queue capacity, so a full queue still drains in a single pass;
// anything arriving during the pass waits for the next one, at most one loop period.
static constexpr int CMD_DRAIN_PER_PASS = CMD_Q_LEN;

static_assert(CMD_DRAIN_PER_PASS == CMD_Q_LEN,
              "the per-pass drain bound must equal the command queue capacity: smaller leaves "
              "work queued for an unbounded number of passes, larger makes the loop's worst "
              "case depend on arrival rate again");
static_assert(CMD_DRAIN_PER_PASS > 0, "a non-positive drain bound would never dequeue");
```

Listing 6.7: The repaired bound and its assertions, `smart_home.ino`

The bound equals the queue capacity, which is the only value that works: a smaller bound leaves work queued for an unbounded number of passes, and a larger one makes the loop's worst case depend on arrival rate again — reintroducing the original defect. The two static assertions exist because deriving one constant from the other is not sufficient protection: a later change could redefine either independently, and the response-time bound holds only while the relationship between them does. Asserting the relationship is preferable to trusting a comment.

6.7.3 Why this defect is the most instructive result in the project

The unbounded drain had survived:

- 129 host tests;
- 3,000,000 fuzzing iterations under AddressSanitizer and UndefinedBehaviorSanitizer;

- 9,600 adversarial trials, including sustained flooding at up to 20,000 messages per second;
- exhaustive model checking of both specifications.

Every one of those techniques exercises a *running system*, and in every execution the loop terminated. None of them asks whether the loop's termination is *bounded*, because that is a question about the program's design rather than about any particular execution. An analytical method costing an afternoon found what several million executions did not.

6.8 The network task

`networkTask()` is a conventional reconnect loop. Its structure is a state check followed by the appropriate action, with each retry governed by its own interval constant so that a failing Wi-Fi connection does not cause an MQTT reconnect storm.

Error handling throughout is by *omission rather than exception*: a failed publish is retried on the next pass, a failed snapshot acquisition is skipped, and a failed NTP synchronisation leaves timestamps uncorrected. None of these conditions can propagate into the safety core, because the only channel into it carries verified commands.

6.9 Event reporting

`emitEvent()` enqueues a structured event on `eventQ`, which the network task drains and publishes as JSON with a sequence number. Events are published oldest first.

The queue drops its *oldest* entry when full rather than refusing the newest. This is the correct choice for a safety device: during an incident the most recent events are the informative ones, and under no circumstances may the safety loop be made to wait for a telemetry consumer.

The event vocabulary is deliberately specific — `cmd_badmac`, `cmd_unauth`, `cmd_replay`, `cmd_rejected`, `cmd_ignored` — and the ablation measurements in Chapter 8 are counted from these names. A coarser vocabulary would have made those measurements impossible.

6.10 Resource cost

Table 5.4 in Chapter 5 gives flash and RAM for both framework versions. The firmware occupies 60.5% of flash under arduino-esp32 2.0.17 and 73.9% under 3.3.12, with RAM at roughly 14–15% in both cases. The headroom is adequate but not generous, which is one reason no additional mechanism was introduced where the repair of an existing one would do.

6.11 How the implementation was tested

Component	How it is tested
State machine	`tests/host/test_firmware.cpp`: transition assertions for every state pair, plus 19,529 observed transitions checked against the TLA+ relation
Input conditioning	host tests for debounce behaviour and the PIR confirmation window

Component	**How it is tested**
Command authentication	differential test D1 in `experiments/test_harness.py`: the firmware's HMAC payload format is compared against an independent implementation for three counter values including `0xFFFFFFFF`
Replay rejection	ablation arms measuring whether captured material yields an accepted command
Build guards	three negative tests that require a defective configuration to *fail* to compile
Drain bound	two `static_asserts`, so a violation is a compile error rather than a runtime surprise
Queue discipline	the `Isolation` TLA+ model, checked exhaustively with no fairness assumption on the network task
Command parser	3,000,000 fuzzing iterations under both sanitizers

The three negative build-guard tests deserve a note. A test that asserts a correct build compiles tells you very little; a test that asserts a *defective* build fails to compile is what demonstrates the guard is doing anything at all. The same principle governs the formal verification in Chapter 9 and the corpus rule controls in Section 8.9.

Chapter 7

Experimental Methodology

7.1 Evidence categories

Results in this project fall into five categories. They are labelled separately throughout because they are not interchangeable, and a reader is entitled to know which kind of claim any particular number is.

Category	**What it is**	**What it does *not* transfer to**
Formal	Exhaustive model checking of the TLA+ specifications	Holds of the *model*. It transfers to the implementation only as far as the trace-conformance check establishes.
Simulation	The unmodified firmware logic driven by a seeded harness on a host. E1–E5.	No radio, no true parallelism, no CPU contention. Carries **no** timing information.
Testbed measurement	Observation of a live external system. E6 only: eight scenarios against a running Mosquitto 2.0.18.	Ran on loopback with a private certificate authority, not in a deployed wireless network.
Analytical	Follows from the construction rather than from running anything: the 2^{-128} forgery probability, the zero blocking term, the response-time decomposition.	Analytical bounds say nothing about the residual hardware term.
Corpus study	Static analysis over 135 commit-pinned open-source projects. E7.	The corpus is biased by construction; the result is an existence claim, never a prevalence estimate.

No result in this project is a physical measurement of the running device. The single exception is the build cost in Table 5.4, measured by a real compiler, which describes the binary rather than the device executing it.

7.2 Hypotheses

Two hypotheses were stated before the experiments were designed.

H1 (logical isolation). No control- or data-flow dependency exists by which network activity can block, delay or corrupt the safety path. *Status: tested, and supported* — by exhaustive model checking of the `Isolation` specification and by 19,529 trace-conformance checks.

H2 (timing isolation). $F_{\Lambda} \stackrel{d}{=} F_0$ for radio-layer load Λ across the operating range. *Status: **requires hardware; not tested here**.* The protocol that would test it is Appendix I.

H2 is the hypothesis that motivated the project, and it is the one that remains open. Nothing in Chapter 8 should be read as evidence for or against it.

7.3 Determinism and reproduction

Every simulated experiment is seeded, and every seed is recorded with the result. The campaign uses seed 1000, the ablation seed 2000, and the fuzzer seed 7. Re-running any harness from a clean checkout regenerates byte-identical records.

Three further measures were taken because reproducibility claims are easy to make and hard to honour:

- **A process per trial.** The campaign harness forks for each trial, so no state can leak between trials through a static variable or an allocator.
- **Standard library only for statistics.** Fisher's exact test and the Wilson interval are implemented in the repository. No external statistical package can alter a reported value, and the Wilson implementation is duplicated independently in two scripts that are checked for agreement.
- **Figures validated against raw records.** `experiments/validate_figures.py` recomputes every plotted value from `ablation.jsonl` rather than from the derived summary, so a figure agreeing with a stale aggregate cannot pass.

7.4 Metrics

Metric	Definition
Safety latency	Simulated milliseconds from confirmed sensor assertion to entry-delay entry. *Simulated, and therefore constant by construction.*
Command availability	Trials in which a legitimate command issued during the flood took effect
Integrity failure	Trials in which an adversary's message caused a state change
Replay defeat	Trials in which the adversary obtained an accepted command using only captured or forged material
Invariant violation	Trials violating any safety invariant
Accepted fuzz input	Inputs changing state or advancing the counter without a valid tag
False-positive rate	Benign runs in which the detector reported at least once
Detection latency	Simulated milliseconds from attack onset to first detector report

7.5 The experiments

7.5.1 E1 and E2: the adversarial campaign

Objective. Determine whether a remote adversary can change the alarm's state or prevent a legitimate operator command from taking effect, with and without command authentication.

Design. 3,000 trials per build across six attack rates from 0 to 20,000 messages per second. The independent variables are the build configuration (baseline or hardened) and the attack rate; the dependent variables are integrity failure, command availability and invariant violation. The rate-0 arms are the controls.

Harness. `experiments/campaign.cpp`, one `fork()` per trial, seed 1000.

7.5.2 E3: the command-path ablation

Objective. Determine which command-path defence preserves the safety function, operator availability and replay resistance — treating the three as separate properties rather than assuming they rise and fall together.

Design. Twelve configurations, 300 trials each, 3,600 trials in total, all on identical seeds. The configurations are not variations on a theme: six are command-path designs and six are credential-compromise scenarios, indexed by which secret the adversary holds.

Mode	What it models
`none`	No command defence at all
`ratelimit`	Token-bucket rate limiting
`token`	A static shared token
`transport`	TLS with credentials held securely
`transport_leaked`	TLS with one credential leaked
`hmac`	HMAC-SHA256 over a monotonic counter
`acl_shared_leaked`	Access control list, one shared credential leaked
`acl_operator_leaked`	Access control list, controller credential leaked
`acl_device_leaked`	Access control list, device credential leaked
`hmac_keyleaked`	HMAC with the command key compromised
`acl_hmac_devicedump`	All three layers, device credential + key (flash dump)
`acl_hmac_-controllerdump`	All three layers, controller credential + key

Harness. `experiments/ablation.cpp`, seed 2000. Broker admission and access control are modelled in `deliverGuarded/admit`; the device-side checks are the firmware's own `handleCommands`.

Availability measurement. Operator availability is measured over *five* attempts per trial at 200 ms intervals, not one. This matters, and the reason is given in Section 7.7.

7.5.3 E4: host-build fuzzing

Objective. Determine whether the command parser mishandles malformed input.

Design. 3,000,000 iterations across six input classes, seed 7, under both AddressSanitizer and UndefinedBehaviorSanitizer.

Scope, stated plainly. The fuzzer exercises the parser *compiled for the host*, not the binary running on the device. That is what makes the sanitizers usable and it is equally the limit of the result: it says nothing about faults depending on the Xtensa toolchain's code generation, on the real memory map, or on the radio driver's own parsing. This is host fuzzing, not firmware rehosting, and the two are not interchangeable [17].

7.5.4 E5: detector parameter sensitivity

Objective. Determine whether the deauthentication detector's shipped threshold and window are appropriate.

Design. A 180-cell grid over threshold and window values, with benign and attack traffic classes. The dependent variables are the false-positive rate and the detection latency.

This experiment produced a correction rather than a confirmation, which is why it is reported rather than dropped.

7.5.5 E6: access control against a live broker

Objective. Determine whether the access-control results modelled in E3 hold against real broker software.

Design. Eight scenarios against a live Mosquitto 2.0.18 instance with TLS 1.2, a private certificate authority, a `mosquitto_passwd` file and a deny-by-default `acl_file`. Two access-control variants are tested — per-client and single-shared-credential — each with its own legitimate control.

The critical methodological decision. Every verdict is decided by whether a subscriber holding the device identity *actually received* the message, never by PUBACK. MQTT 3.1.1 has no return code meaning "refused by policy", so a broker that discards a publication for violating the access control list still acknowledges a QoS 1 publish. The summary deliberately prints the PUBACK column in order to show that it is uninformative.

Limitation. The broker ran on loopback with a private certificate authority, not in a deployed wireless network.

7.5.6 E7: the open-source corpus study

Objective. Determine whether the configuration defect found in this system appears in comparable open-source projects.

Design. 135 projects selected by 22 fixed GitHub queries with a minimum of 3 stars and a maximum size of 80 MB, excluding archived repositories and forks. Every project is pinned to a commit hash recorded in `commit_shas.tsv`. Two static rules are applied: a precise guard pattern (Rule 1) and a looser no-application-authentication heuristic (Rule 2).

Controls. Eight, including this project's *own* pre-repair guard as a positive control and its repaired guard as a negative control. The reason is given in Section 7.7.

Bias, stated up front. The corpus is biased by construction. Query selection, the star threshold and the size cap all shape it. Whatever the result, it cannot support a prevalence claim about the wider ecosystem.

7.5.7 E8 and E9: conformance and model checking

E8, trace conformance. The transition relation is parsed out of `formal/AlarmFSM.tla` — rather than restated — and 19,529 firmware transitions observed across three ablation modes are checked against it. A negative control asserts that a forbidden `Disarmed` → `Alarm` transition *is* rejected.

E9, model checking. Three TLC configurations: `AlarmFSM`, `Isolation`, and a deliberately defective `AlarmFSM_unsafe` which the runner requires to *fail*. Chapter 9 explains why the third exists.

7.6 The experimental setup as a whole

Figure 7.1 distinguishes what has been run from what has only been specified.

7.7 Defects found in the measurement infrastructure

Four defects were found in the harnesses themselves during this work. Two of them produced results that flattered the intended conclusion, which is precisely why they are reported here rather than quietly corrected. A reader assessing the credibility of Chapter 8's numbers is entitled to know how the instrument was tested.

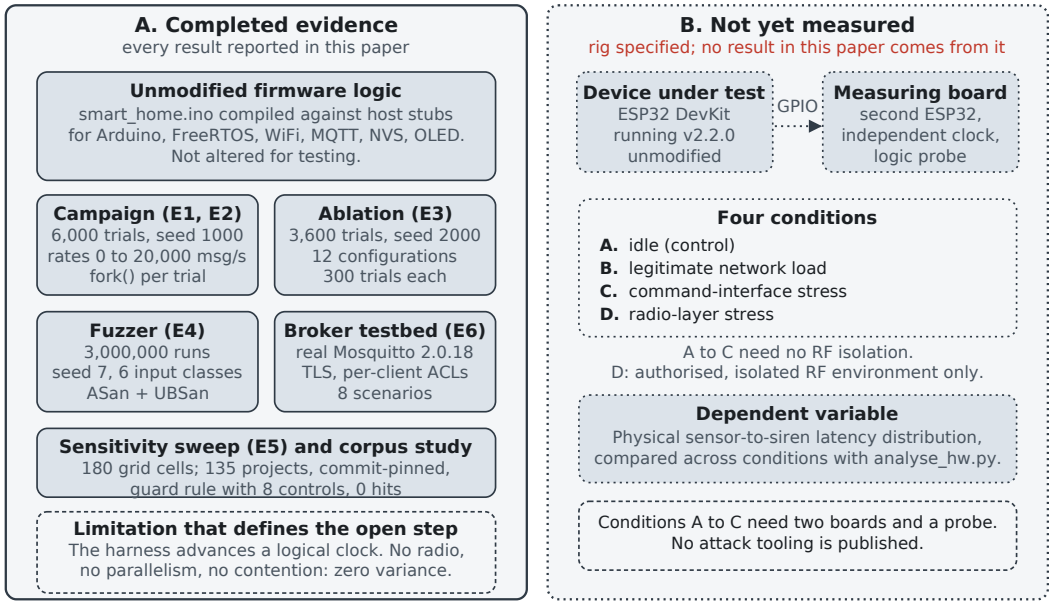


Figure 7.1: Experimental setup. Panel A is everything that has been run: the host simulation harness (E1–E5) and the real-broker testbed (E6), which is a live Mosquitto instance and not a simulation. Panel B is the hardware rig: specified, built as a protocol, and **not run**. No number anywhere in this report comes from panel B.

1. **Operator availability was unattributable.** The original measurement recorded whether a legitimate command took effect, but the adversary in the harness could also publish `ARM` — so an observed state change could not be attributed to the operator. Fixed by suppressing adversary `ARM` publications for the whole measurement prologue.

2. **The first fix was itself wrong.** The quiet settling period introduced by that fix allowed the token bucket to refill before the measurement, so the rate limiter appeared more available than it is. Fixed by holding the suppression across the entire prologue rather than part of it.

3. **A single-attempt measurement measured the wrong thing.** Availability recorded as one boolean per trial was in fact measuring the token bucket's *phase* at the instant of the attempt, producing residues of 0.5, 0.9 and 0.995 rather than a rate. Fixed by measuring five attempts per trial at 200 ms intervals, which is why the availability denominator is 1,250 and not 300.

4. **A button hold shorter than the debounce window.** The harness held the simulated button for 40 ms against a 40 ms debounce threshold, so local arming was intermittent for reasons unrelated to the experiment. Fixed to 80 ms.

The campaign was re-run from scratch after these corrections, and every number in Chapter 8 comes from the corrected harness.

A fifth defect, in the corpus rule rather than the harness, is reported in Section 8.9: the rule initially returned zero against this project's own pre-repair guard, because the identifier it looked for appears there *negated*. Until that was found, the zero result was meaningless. The eight controls exist as a direct consequence.

Chapter 8

Results and Statistical Analysis

Every number in this chapter was recomputed from the raw per-trial records during the preparation of this report, not copied from an earlier document. The recomputation script is `experiments/validate_figures.py`, which reads `ablation.jsonl` rather than the derived summary.

8.1 Statistical treatment

Three decisions govern every test reported below.

Interval estimates. Proportions are reported with 95% Wilson score intervals rather than the normal approximation, because several arms sit at or near $p = 0$ or $p = 1$ where the normal interval is degenerate and can extend outside $[0, 1]$. Intervals are clamped at the unit bounds.

Hypothesis tests. Comparisons of two proportions use Fisher's exact test, one-sided, in preference to χ^2: several of the 2×2 tables contain zero cells, where the χ^2 approximation is not valid.

Where no test is reported. No hypothesis test is applied to a quantity whose variance is identically zero by construction. The simulation harness advances a logical clock, so the safety latency is the same constant — 151 ms, being the sensor confirmation window plus the loop period — in every trial. A test on two constant-valued samples is trivially satisfied and carries no information. The pre-registered test for the hardware measurement is in Appendix I and was committed before any data existed.

Because every arm is a fixed, pre-declared configuration rather than a selected subset, no multiplicity correction is applied. This is noted so a reader can judge it rather than infer it.

8.2 Trial accounting

ID	Experiment	Scale	Artefact
E1	Campaign, baseline build	3,000 trials	`summary.json`
E2	Campaign, hardened build	3,000 trials	`summary.json`
E3	Command-path ablation	3,600 trials	`ablation.jsonl`
E4	Host-build fuzzing	3,000,000 iter.	`fuzz.json`
E5	Detector sensitivity	180 grid cells	`sensitivity.jsonl`
E6	Live-broker access control	8 scenarios	`broker_acl.jsonl`
E7	Open-source corpus	135 projects	`corpus.json`
E8	Trace conformance	19,529 transitions	`test_conformance.py`
E9	Model checking	25 and 660 states	`formal/`
Total adversarial trials		**9,600**	3,000 + 3,000 + 3,600

8.3 E1 and E2: the adversarial campaign

Across 6,000 trials **no adversary disarmed the device and no safety invariant was violated.**

Without command authentication the adversary changed the system state in all 2,500 flooded trials; with authentication, in none. The availability result is the more informative of the two: with authentication a legitimate command took effect in all 3,000 trials regardless of attack rate, while in the baseline, flooding starved it in every flooded trial.

What this does not show. It shows nothing about timing. The safety latency is a constant in this environment, so the campaign establishes that the *logic* holds under adversarial load and nothing about whether the *silicon* does.

8.4 E3: the command-path ablation

8.4.1 The six command-path designs

Figure 8.1 shows three properties across the six designs.

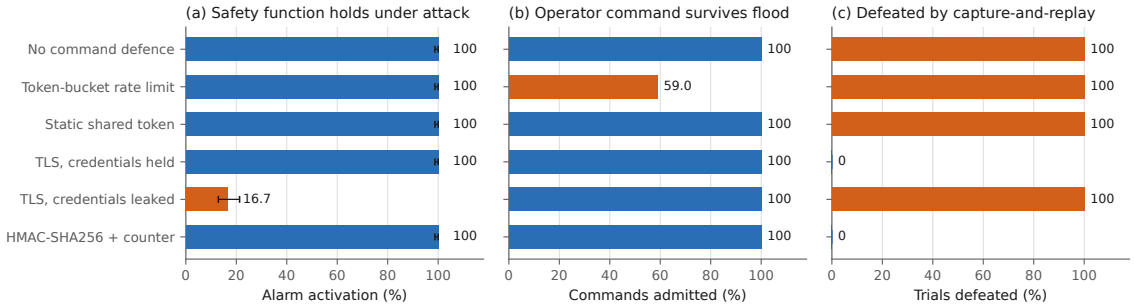


Figure 8.1: Command-path ablation. 300 trials per bar in panels (a) and (c); 1,250 operator attempts per bar in panel (b). Whiskers in (a) are 95% Wilson score intervals. Panel (c) inverts the reading: there a high value is the failure, so the colour rule inverts with it, and blue continues to mean "this design holds the property".

Design	Alarm	95% Wilson	Avail.	Replay
No command defence	300/300	[98.7, 100.0]	100.0%	100%
Token-bucket rate limit	300/300	[98.7, 100.0]	**59.0%**	100%
Static shared token	300/300	[98.7, 100.0]	100.0%	100%
TLS, credentials held	300/300	[98.7, 100.0]	100.0%	0%
TLS, credentials leaked	**50/300**	**[12.9, 21.3]**	100.0%	100%
HMAC-SHA256 + counter	300/300	[98.7, 100.0]	100.0%	0%

8.4.2 Three findings

The three properties fail independently. This is the central result of the ablation and it is not what one would assume. The rate limiter preserves the safety function and replay resistance is unchanged at 100% defeated, yet it costs **41% of legitimate operator commands**. The static token does the reverse: every operator command is admitted and replay resistance is nil. Only the counter-bound message authentication code holds all three. A designer who treats "security" as one property will choose badly here.

The availability difference between the rate limiter and the others is significant at Fisher's exact test, one-sided, $p = 3.5 \times 10^{-184}$.

Transport security alone is contingent on credential secrecy. With credentials held, TLS gives 100% alarm activation and no replay. With a single credential leaked, alarm activation falls to 16.7%, 95% Wilson [12.9, 21.3], and the adversary disarms the device in 246 of 300 trials. The difference against the credentials-held arm is significant at $p = 9.8 \times 10^{-119}$.

The alarm still firing is not the same as the system being secure. In the `none`, `ratelimit` and `token` arms the alarm fires in all 300 trials, and all three are completely defeated by capture-and-replay. Panel (a) read alone would suggest that no command defence is as safe as a message authentication code. Panel (c) is what prevents that reading, and this is why the figure has three panels rather than one.

8.5 Broker authorisation against message authentication

8.5.1 The nine-cell matrix

Figure 8.2 indexes outcomes by *which secret the adversary holds* rather than by mechanism, because that is the comparison the design question requires.

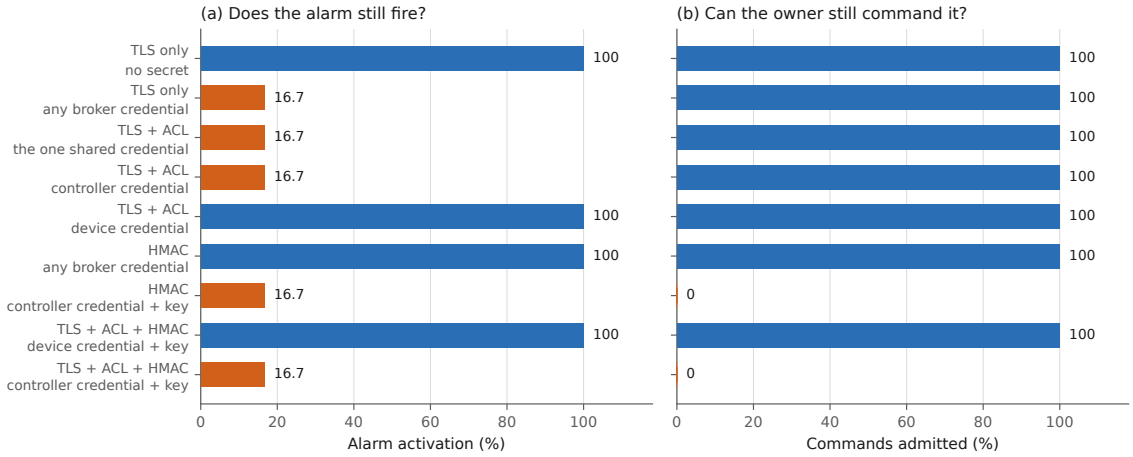


Figure 8.2: Alarm activation and operator availability for nine combinations of control and compromised secret. The bottom row is the layered build under a compromised controller host: it fails, and is drawn at the same weight as every other row.

Control	Compromise	Alarm	Operator admitted	Disarms
TLS only	none	300/300	1250/1250	0
TLS only	any broker credential	50/300	1250/1250	246
TLS + ACL	the one shared credential	50/300	1250/1250	246
TLS + ACL	controller credential	50/300	1250/1250	246
TLS + ACL	device credential	300/300	1250/1250	0
HMAC + counter	any broker credential	300/300	1250/1250	0
HMAC + counter	controller cred. + key	50/300	**0/1250**	234
TLS+ACL+HMAC	device cred. + key	300/300	1250/1250	0
TLS+ACL+HMAC	controller cred. + key	50/300	**0/1250**	234

8.5.2 Neither control dominates

The matrix shows that broker authorisation and application-layer authentication fail under *different* compromises, and that neither contains the other.

The access control list contains a compromised *device* credential, which the message authentication code does not — because the command key lives in the same flash image as the device's broker credential, so a flash dump yields both. Conversely the message authentication code contains a compromised *controller* credential, which the access control list cannot — because the list must permit that identity to publish commands.

8.5.3 An unanticipated negative result: owner lockout

Two rows show **0 of 1,250** operator commands admitted. This was not predicted.

Under key compromise the adversary signs commands with a counter far above the operator's. The device persists that counter, as it must in order to resist replay. Every subsequent legitimate command then carries a lower counter and is correctly rejected as a replay. The mechanism that defeats capture-and-replay has become a lockout against the legitimate owner.

The mechanism is not new; the measurement is. A literature check carried out for this report found the mechanism stated as background in work on ZigBee replay attacks: Wara and Yu [23] note that an attacker can set a device's frame counter above its normal value so that subsequent lower-counter frames are discarded. They state it in passing, without a citation and without an experiment, and their own measurements go the other way — they test whether raising a captured frame's counter lets a *replay* succeed, not whether it blocks legitimate traffic.

What this work adds is therefore not the observation but the quantification and the attribution: the lockout is complete rather than partial (0 of 1,250), it arises from a design decision made for a defensible reason, and it shows that replay resistance and owner availability are coupled under key compromise rather than independent. An earlier draft of this report claimed the mechanism had not been reported at all. That claim was wrong and has been withdrawn.

8.5.4 An overclaim found by measuring the missing arm

An earlier draft of this work stated that the layered configuration "survives every case". That statement was false, and it was found to be false by running the arm the draft had not run.

The layered configuration survives every single-secret compromise *and* a device flash dump. It falls to a compromised controller host. Since the flash dump requires physical access and the controller-host compromise does not, the configuration survives the expensive scenario and fails the cheap one. Figure 8.3 draws both.

8.6 E6: results against a live broker

All eight scenarios were run against Mosquitto 2.0.18 over TLS 1.2 with a private certificate authority. The verdicts reproduce: re-running the experiment on a freshly installed broker in a clean environment during the preparation of this report returned verdicts identical to the committed run, with only the broker's own log timestamps and random client identifiers differing.

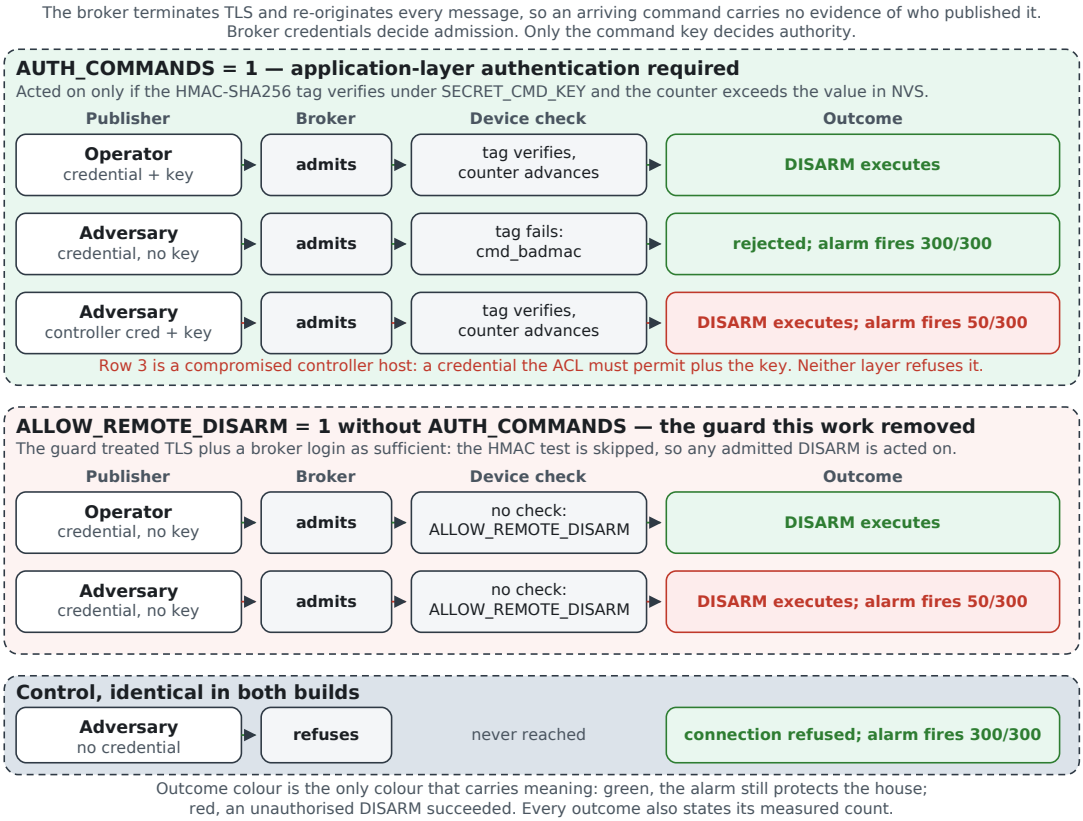


Figure 8.3: Where broker credentials decide admission and where the command key decides authority. The third row of the upper panel is the compromised controller host, holding both a permitted credential and the key: neither layer refuses it, and the alarm fires in only 50 of 300 trials.

	Adversary identity	Connected	Delivered	Would execute
Per-client access control list				
control	controller (legitimate)	yes	yes	yes
A	no credentials	**no**	no	no
C	controller credential	yes	yes	**yes**
D	device credential	yes	**no**	no
E	device credential + key	yes	**no**	no
E′	controller credential + key	yes	yes	**yes**
Single shared credential				
control	controller, shared ACL	yes	yes	yes
B	shared credential	yes	yes	**yes**

E and E′ are the decisive pair. The *same valid signature*, carried by the device identity, is not delivered; carried by the controller identity, it is. What stops scenario E is therefore the access control list and not the signature. This is the result that could not be established by modelling alone, and it is the reason the experiment was worth running against real broker software.

8.7 E5: detector parameter sensitivity

The sweep produced a correction rather than a confirmation. At the detector's original shipped configuration — threshold 8 frames within a 2,000 ms window — the detector fired in **79.9%** of benign ten-minute runs (1,597 of 2,000). That is not a detector; it is an alarm that reports almost continuously under normal conditions.

The firmware now ships `DEAUTH_THRESHOLD = 12`. The sweep is reported because the original value was wrong by a wide margin and the sweep is what revealed it.

8.8 E4: fuzzing

3,000,000 iterations across six input classes under both sanitizers produced **no crash, no sanitizer report and no accepted input**: no input changed state or advanced the counter without a valid tag.

What this is evidence of, and what it is not. It is evidence of memory safety and parser robustness. It is *not* independent evidence of authentication soundness: against a 128-bit tag, zero forgeries in three million attempts is the expected outcome whether the implementation is sound or not, since the probability of a chance forgery is 2^{-128}.

8.9 E7: the corpus study, and a negative result

8.9.1 Rule 1 found nothing

The precise guard rule returned **zero hits across 135 projects**.

That is reported as a zero. **No claim is made that the pattern is widespread**, and the finding remains a single-system case study.

The zero is also explained rather than left bare. Only 30 of the 135 projects both subscribe to MQTT and contain any application-authentication identifier at all, and reading those shows that most matches sit in vendored third-party code rather than in the project's own command path. The corpus is biased by construction — 22 fixed queries, a minimum

of 3 stars, a size cap of 80 MB, archived repositories and forks excluded — and those criteria are recorded with the rejection counts in `corpus.json`.

8.9.2 The rule was wrong at first

This matters for the credibility of the zero. The rule initially returned zero against *this project's own pre-repair guard*, because it treated any mention of the authentication identifier as evidence that authentication was required — and in the defective guard that identifier appears *negated*. Until this was found, the zero result was meaningless.

Eight controls now exist as a direct consequence, including the project's own pre-repair guard as a positive control and its repaired guard as a negative control. All eight pass.

8.9.3 Rule 2 and the nine confirmed projects

The looser heuristic flagged 18 of 135. Reading all 18 by hand gave 9 confirmed and 9 false positives.

This is reported as an **existence claim and not a rate**. The rule over-counts monitoring projects by roughly half and under-counts non-GPIO actuation by an unknown amount: three of the nine confirmed cases actuate through Modbus, RF or serial frames and were caught only by an incidental GPIO write. The 18/135 figure is therefore not a prevalence estimate and must not be read as one.

8.10 E8 and E9: conformance and model checking

Trace conformance. 19,529 firmware transitions across three ablation modes — 7,044, 5,282 and 7,206 — checked against the transition relation parsed from `AlarmFSM.tla`. **Zero disallowed transitions.** The negative control confirms that a forbidden `Disarmed` → `Alarm` transition *is* rejected, which is what makes the 19,529 meaningful: a checker that accepts everything would also report zero violations.

Model checking. `AlarmFSM`: 166 states generated, 25 distinct. `Isolation`: 2,686 generated, 660 distinct. Both exhaustive. The third configuration and its purpose are the subject of Chapter 9.

8.11 Summary of statistical values

Every value below was recomputed from the raw records during the preparation of this report.

Claim	Recomputed	Status
Leaked-credential alarm activation 16.7%	50/300 = 16.67%	exact
95% Wilson interval [12.9%, 21.3%]	[12.9, 21.3]	exact
Rate limiter admits 59.0% of operator commands	737/1250 = 59.0%	exact
Fisher exact, rate limiter vs others	3.53×10^{-184}	exact
Fisher exact, transport leaked vs held	9.83×10^{-119}	exact
Adversary disarms in 246 of 300 trials	246	exact
Owner lockout 0 of 1,250	0/1250	exact
Detector default fired in 79.9% of benign runs	1597/2000 = 79.85%	exact
TLC distinct states, 25 and 660	25 and 660	exact

Claim	**Recomputed**	**Status**
Trace transitions, 19,529	7044+5282+7206	exact

Chapter 9

Formal Verification

Two TLA+ specifications were written for this project and are checked exhaustively by TLC on every push. This chapter states what they establish, and — with equal emphasis — what they do not, because the second specification is named `Isolation` and that name invites a stronger reading than the model supports.

The chapter also records a defect in the first specification. For part of this project the specification passed every check while being incapable of failing. That is reported here rather than quietly repaired, because it is the most instructive methodological result in the work.

9.1 Why formal methods were applied at all

The firmware's safety claim is universal: *no* sequence of adversarial messages causes an unauthorised disarm. Testing cannot establish a universal claim over an unbounded input space; it can only fail to refute it. The adversarial campaign of Chapter 8 ran 9,600 trials and found no violation, which is evidence but not proof. Exhaustive model checking over a bounded abstraction is the complementary instrument: it enumerates every reachable state of the abstraction and therefore *proves* the invariant within that abstraction's bounds.

The two instruments fail in different directions, which is why both are used. The model may abstract away the very detail that breaks in reality; the experiment covers reality but only the paths it happened to take.

9.2 Specification 1: the alarm state machine

9.2.1 What is modelled

`formal/AlarmFSM.tla` models the five-state arm/disarm machine and the authenticated command path, over three variables:

- `state` — one of `Disarmed`, `ExitDelay`, `Armed`, `EntryDelay`, `Alarm`
- `ctr` — the highest accepted authenticated command counter
- `badDisarm` — set `TRUE` if an unauthenticated remote disarm ever takes effect

Time is abstract. A delay expires as a nondeterministic step rather than by counting milliseconds. This is the correct level of abstraction for reasoning about *which transitions are reachable and who may cause them*, and it is the wrong level for reasoning about *how long* anything takes. No timing conclusion is drawn from this model anywhere in this report.

9.2.2 Properties checked

Property	Kind	Statement
`TypeOK`	invariant	All three variables stay in their declared domains

Property	Kind	Statement
`NoUnauthRemoteDisarm`	invariant	`badDisarm = FALSE`: the system never leaves an armed state because of an unauthenticated remote command
`NoAlarmWhenDisarmed`	invariant	The siren is never on while disarmed
`CounterMonotonic`	temporal	$[][ctr' \geq ctr]$: the replay counter never decreases
`EntryLeadsToResolve`	liveness	An entry delay always ends in `Alarm` or `Disarmed`; the system cannot get stuck mid-entry
`AlarmReArms`	liveness	The alarm always ends, in re-arm or disarm

The liveness properties require fairness on the three timed transitions (`ExitExpire`, `EntryExpire`, `AlarmExpire`), supplied by weak fairness in `Spec`. No fairness is assumed for any adversary action.

9.2.3 Result

With `MaxCtr = 4` and `AllowUnauthDisarm = FALSE`: **166 states generated, 25 distinct, 0 left on queue, no error found.** The run is exhaustive over the bounded state space, and both temporal branches were checked over the complete space.

9.3 The vacuity defect, and the repair

9.3.1 What was wrong

In the original specification the unauthenticated-command action was written as an unconditional refusal:

```
1 \* An UNauthenticated remote command is refused, exactly as the firmware does.
2 RemoteUnauthDisarm ==
3   /\ state \in ArmedLike
4   /\ UNCHANGED <<state, ctr, badDisarm>>
```

Listing 9.1: The defective action, as originally written (reconstructed from the repair commit).

The comment asserts the conclusion. `badDisarm` was initialised `FALSE` and left `UNCHANGED` by *every* action in the specification, so no reachable behaviour could set it. The invariant `NoUnauthRemoteDisarm` was therefore **unfalsifiable by construction**: not proved, but incapable of failing.

TLC reported success. The success meant nothing.

9.3.2 Why this mattered more than an ordinary bug

The firmware contained a real defect of exactly the kind this invariant was written to catch: a compile-time guard that admitted remote disarm without application-layer authentication. The ablation experiment found it. The model checker, which existed specifically to find it, could not have found it under any circumstances.

The specification had embedded the same unverified assumption that the firmware got wrong. A passing formal check had been taken as evidence for a property that the check was structurally unable to test. This is the project's central methodological finding, generalised in Chapter 13.

9.3.3 The repair

The build switch is now a model parameter rather than a hard-coded refusal:

```
RemoteUnauthDisarm ==
  /\ state \in ArmedLike
  /\ IF AllowUnauthDisarm
       THEN /\ state' = "Disarmed"
            /\ badDisarm' = TRUE
            /\ UNCHANGED ctr
       ELSE UNCHANGED <<state, ctr, badDisarm>>
```

Listing 9.2: The repaired action: the configuration is a parameter, so the violating behaviour exists and is reachable under one valuation.

9.3.4 The third configuration: a check that must fail

The repair alone is not sufficient, because a future edit could reintroduce the vacuity and the suite would again report success. A third configuration therefore exists whose *required outcome is a violation.*

`AlarmFSM_unsafe.cfg` sets `AllowUnauthDisarm = TRUE`, modelling the `ALLOW_REMOTE_DISARM` build with no application-layer authentication. `formal/run_tlc.sh` inverts the exit condition for this run and additionally requires the specific string `Invariant NoUnauthRemoteDisarm is violated` in TLC's output:

```
if java -cp "$JAR" tlc2.TLC -config AlarmFSM_unsafe.cfg AlarmFSM.tla > /tmp/tlc_unsafe.out 2>&1
then
  echo "FAIL AlarmFSM_unsafe: TLC found no violation, so the invariant is vacuous again"
  fail=1
elif grep -q "Invariant NoUnauthRemoteDisarm is violated" /tmp/tlc_unsafe.out; then
  echo "PASS AlarmFSM_unsafe: TLC reports the expected NoUnauthRemoteDisarm violation"
```

Listing 9.3: The inverted check. A *passing* TLC run is treated as a build failure.

Requiring the specific invariant name, rather than merely a non-zero exit, matters: a syntax error, a missing constant or a parse failure also produce a non-zero exit, and would otherwise be silently accepted as the expected violation.

TLC reports the expected violation on every run. The invariant is now known to be falsifiable, which is what licenses reading its success in the safe configuration as evidence.

9.4 Specification 2: the isolation model

9.4.1 What is modelled

`formal/Isolation.tla` models the two-task structure over seven variables: `safetySteps`, `produced`, `queued`, `delivered`, `dropped`, `netState` and `netOwnsState`.

The network task is modelled **adversarially and with no fairness whatsoever**. It may move between `idle`, `busy`, `stalled` and `down`; it may stall for an unbounded time, as in a slow DNS, TCP or TLS call; it may fail to publish; and it may crash and restart. Only the safety task is given weak fairness. The question the model answers is whether the safety task still progresses when the network task is permitted to behave as badly as it likes forever.

9.4.2 Properties checked

Property	Kind	Statement
`TypeOK`	invariant	All seven variables stay in their domains
`SafetyCanAlwaysStep`	invariant	$(safetySteps < MaxEvents) \Rightarrow$ ENABLED *SafetyStep* — in every reachable state the safety task is enabled
`NoNetworkWrite`	invariant	`netOwnsState = FALSE`: the network task never writes the alarm state
`QueueBounded`	invariant	$queued \leq QCap$: the writer drops instead of blocking
`NoSilentLoss`	invariant	$produced = delivered + queued + dropped$
`SafetyProgress`	liveness	$\Diamond(safetySteps = MaxEvents)$: the safety task is never starved

`SafetyCanAlwaysStep` is the architectural claim stated formally. Its force comes from what the guard of `SafetyStep` mentions: `safetySteps` and `produced` only, and nothing owned by the network side. Because the enabling condition is independent of `netState`, no network behaviour can disable it. `NoSilentLoss` is the drop-oldest policy stated as an accounting identity: an event is delivered, still queued, or counted as dropped, and never disappears.

9.4.3 Result

With `QCap = 16` and `MaxEvents = 8`: **2,686 states generated, 660 distinct, 0 left on queue, no error found**, with the temporal property checked over the complete 660-state space.

9.4.4 What this model does and does not establish

This distinction is the most important sentence in the chapter, and it is maintained verbatim throughout this report and the manuscript.

What it establishes. *Logical* isolation. The safety task's enabling condition is independent of every network-side variable, so there exists no reachable state — under unbounded stalling, flooding, publication failure or crash — in which the network side has disabled the safety side. In the firmware this corresponds to the asymmetric mutex timeout: the safety loop calls `xSemaphoreTake(statusMutex, 0)`, so its blocking term is identically zero, by construction and not by measurement.

What it does not establish. *Physical timing invariance.* The model has no notion of microseconds, cache lines, bus arbitration, interrupt latency, radio duty cycle, or the thermal and clock behaviour of a real die. Two cores on one package contend for shared silicon in ways a seven-variable state machine cannot represent. Logical isolation does not imply that the safety loop's measured wall-clock period is invariant to network load on real hardware, and nothing in this project has measured whether it is.

The firmware is designed so that the *blocking* component of that period is zero. Whether the *total* period is invariant is an empirical question about silicon, requiring hardware conditions A–D. Those were not executed; Appendix I records why, and the pre-registered analysis plan that was committed before any data existed.

9.5 Trace conformance: connecting the model to the code

A specification can be correct about a machine the firmware does not implement. Trace conformance is the bridge.

`experiments/test_conformance.py` parses the transition relation directly from `AlarmFSM.tla` — rather than re-encoding it in Python, where it could drift — and checks every recorded firmware state transition against it.

19,529 transitions across three ablation modes: 7,044, 5,282 and 7,206. **Zero disallowed transitions.**

The figure is meaningful only because of the negative control. The same checker is given a deliberately forbidden transition, `Disarmed` → `Alarm`, and is required to reject it. It does. Without that control, a checker that accepted everything would also report zero violations across 19,529 transitions, and the result would be indistinguishable from the one reported here. This is the same reasoning as the inverted TLC configuration in Section 9.3, applied to a different instrument.

9.6 Summary

Configuration	Result	Required outcome
`AlarmFSM.cfg`	25 distinct states, no error	pass
`Isolation.cfg`	660 distinct states, no error	pass
`AlarmFSM_unsafe.cfg`	`NoUnauthRemoteDisarm` violated	**violation required**; a pass is a build failure
`test_conformance.py`	19,529 transitions, 0 violations	pass, with the negative control rejected

All four are re-run by continuous integration on every push, so neither the vacuity defect nor a silently accepting conformance checker can recur without the build failing.

Chapter 10

Security Evaluation

This chapter evaluates the system against the threat model of Chapter 3 and reports two results that the project did not set out to find: that the layered configuration fails under one specific compromise, and that the mechanism defending against replay can lock out the legitimate owner. Both are reported as findings about this design rather than as limitations of the evaluation.

10.1 The threat model, restated with outcomes

Each scenario is defined by *what the adversary holds*, not by what mechanism is deployed. The access column is material: it separates a scenario requiring physical possession of the device from one that can be mounted from anywhere on the network.

ID	Adversary holds	Access	Outcome against the layered build
A	nothing	remote	Refused at the broker; the connection is not established
B	the one shared credential	remote	**Defeats** a shared-credential deployment; refused by per-client authorisation
C	controller broker credential	remote	Delivered; refused by the message authentication code
D	device broker credential	remote	Refused by the access control list; the device identity may not publish commands
E	device credential + command key	**physical**	**Survived**: refused by the access control list despite a valid signature
E′	controller credential + command key	**remote**	**Defeats the layered build**: neither layer refuses it

10.1.1 Why E and E′ are the decisive pair

Scenarios E and E′ carry *the same cryptographically valid signature.* They differ only in the broker identity that transmits it.

Under E the signature arrives from the device identity. The access control list grants that identity `read` on `home/alarm/+/cmd` and `write` on `home/alarm/+/stat`; it does not grant `write` on the command topic. The broker refuses the publication, so the valid signature never reaches the device.

Under E′ the same signature arrives from the controller identity, which must hold `write` on the command topic for the system to function at all. The broker delivers it, and the message authentication code verifies — correctly, because the signature is valid. The command executes.

What stops scenario E is therefore the access control list and not the signature. This cannot be established by reasoning about the cryptography, and it is the result that justified running experiment E6 against real broker software rather than modelling broker behaviour. Both scenarios were executed against Mosquitto 2.0.18 over TLS 1.2 with a private certificate authority; the verdicts in Section 8.6 are the broker's own.

10.1.2 An asymmetry in cost

E requires physical possession of the device and a flash dump. E′ requires compromise of the controller host — a general-purpose machine, typically a Raspberry Pi or a server, reachable over the network and administered by software with its own vulnerability history.

The layered configuration therefore **survives the expensive attack and falls to the cheap one.** That is the opposite of the ordering a defender would choose, and it is stated plainly because an earlier draft of this work did not state it.

10.2 Flash extraction and what is actually protected

10.2.1 The threat

The ESP32's SPI flash can be read over the serial interface with standard vendor tooling unless flash encryption and secure boot are enabled. This firmware does not enable them. Anyone with physical possession of the device can therefore recover the entire image, which contains both the device's broker credential and `SECRET_CMD_KEY`.

10.2.2 Why the device survives it anyway

This is a genuine positive result and is worth stating precisely, because it is easy to overstate.

The device survives a flash dump *not* because the key is protected — it is not — but because **the identity whose secrets were extracted is not authorised to issue commands.** The access control list is deny-by-default and grants the device identity publication rights only on the status topic. An adversary holding everything in the device's flash can sign a valid `DISARM` command and still cannot get it accepted, because the broker refuses to carry it.

This is a defence-in-depth result in the strict sense: a complete failure of secret confidentiality at one layer is contained by authorisation at another.

10.2.3 What it does not mean

It does not mean key extraction is harmless. It means key extraction *from the device* is contained. Key extraction from the controller host is scenario E′, which is not contained. The protection comes from *which* identity holds the key, not from the key being hard to obtain.

Three consequences follow and are recorded in Chapter 12: a deployment that issues one credential to both parties collapses E′ and E into the same scenario and loses the protection entirely; a deployment whose access control list grants the device identity write access on the command topic, for convenience during commissioning, loses it as well; and flash encryption with secure boot would address the extraction itself rather than only containing it.

10.3 The compile-time guard

10.3.1 The original defect

The firmware originally admitted remote disarm when the build had transport security and a broker login, on the reasoning that a client which has authenticated to the broker over TLS is trustworthy. That reasoning is wrong for a specific and documented reason: in MQTT 3.1.1 the broker *re-originates* messages. The subscriber receives a message from the broker, not from the publisher, and the protocol carries no publisher identity to the

subscriber. Transport authentication tells the broker who connected; it tells the device nothing about who authored a command.

Broker credentials are bearer tokens. One leaked credential makes the adversary a legitimate client, and the ablation measured the consequence: alarm activation falls from 300/300 to 50/300, with the adversary disarming the device in 246 of 300 trials (Section 8.4).

10.3.2 The repair, and why it is a build-time error

The defective configuration is now rejected at compile time:

```
#ifndef ALLOW_REMOTE_DISARM
#  define ALLOW_REMOTE_DISARM 0
#endif
#if ALLOW_REMOTE_DISARM && !AUTH_COMMANDS && !defined(ACKNOWLEDGE_UNAUTHENTICATED_DISARM)
#  error "ALLOW_REMOTE_DISARM without AUTH_COMMANDS 1 permits anyone who can publish to the \
command topic to switch the alarm off, and TLS plus a broker login does not prevent it (see \
docs/SECURITY_ANALYSIS.md). Set AUTH_COMMANDS 1, or define \
ACKNOWLEDGE_UNAUTHENTICATED_DISARM to accept that risk deliberately."
#endif
```

Listing 10.1: The build guard. The configuration cannot be built accidentally.

Three design decisions in that guard are deliberate.

It is an error, not a warning. A warning in an Arduino build is lost in several hundred lines of framework output.

The escape hatch is named for what it concedes. The macro is `ACKNOWLEDGE_UNAUTHENTICATED_DISARM`, not `FORCE` or `OVERRIDE`. A builder who defines it has written the concession into their build command.

The escape hatch exists. The defective configuration must remain buildable, because it is the research baseline. Deleting it would make the measured comparison irreproducible. This is the one place in the firmware where a known-insecure configuration is retained on purpose, and the reason is recorded next to it.

10.3.3 A second guard: the published key

A related defect was found and guarded during the same review. Authentication that verifies against a published key is not authentication. The repository's `SECRET_CMD_KEY` default is a public demonstration string, so a build enabling `AUTH_COMMANDS` without overriding it would log every command as authenticated while any reader of the repository could forge one. That configuration is now also rejected at build time.

Three negative build tests in `tests/host/run_tests.sh` require each defective configuration to *fail to compile*. A build that succeeds is a test failure.

10.4 Availability as a security property

The ablation treats operator availability as a measured security property rather than a usability concern, and two arms show why.

The rate limiter. A token bucket on the command path preserves the safety function and leaves replay resistance unchanged, and costs **41% of legitimate operator commands** (737 of 1,250 admitted, Fisher exact $p = 3.5 \times 10^{-184}$). An alarm that refuses two in five owner commands is not a secure alarm; it is a denial of service the defender installed.

Owner lockout under key compromise. Two configurations admit **0 of 1,250** operator commands. The adversary signs commands with a counter far above the operator's; the device persists that counter, as replay resistance requires; every subsequent legitimate command then carries a lower counter and is correctly rejected as a replay.

The mechanism behaved exactly as specified. The specification was incomplete: it did not consider that an adversary who can advance the counter can also exhaust the operator's ability to use it. This was not predicted by this project, though the mechanism itself is not new: Wara and Yu [23] state it as background in the ZigBee setting, without a citation and without an experiment. What is contributed here is the measurement — a complete rather than partial lockout, 0 of 1,250 — and the resulting observation that replay resistance and owner availability are coupled under key compromise. It is reported as a negative result about a design decision made in this project, and a candidate repair — a counter-resynchronisation path authenticated by the local button, which is physically present and outside the adversary's reach — is identified in Chapter 12 as future work. It has not been implemented or evaluated.

10.5 Properties established, and by which instrument

Property	Evidence	Scope of the claim
No unauthorised disarm with command authentication	6,000 trials + exhaustive model checking	Within the harness's adversary model and the model's bounds
Replay resistance	3,600 trials, 0 successes	Against capture-and-replay of observed ciphertext
Memory safety of the parser	3,000,000 fuzz iterations under both sanitizers	Host build; not the Xtensa target
Flash dump contained	live broker, scenario E	Contingent on per-client authorisation and on the device identity lacking command write access
Logical isolation of the safety task	exhaustive model checking, 660 states	Logical only; **no physical timing claim**
Specification conformance	19,529 transitions, 0 violations	Three ablation modes, with a negative control

10.6 Properties explicitly not established

- **Physical timing invariance.** Not measured. Hardware conditions A–D were not executed (Appendix I). Every isolation claim in this work is logical.
- **On-device memory safety.** The fuzzing campaign ran against a host build with AddressSanitizer and UndefinedBehaviorSanitizer, neither of which exists for the Xtensa target. Memory safety of the compiled Xtensa binary is not established.
- **Resistance to radio-layer attack.** No RF-shielded enclosure was available, so no deauthentication or jamming experiment was performed against real hardware.
- **Resistance to side-channel or fault-injection attack.** Out of scope; no equipment.
- **Prevalence of the guard defect in deployed systems.** The corpus study returned zero hits for the precise rule and nine confirmed cases for the looser one. That is an existence claim over a biased 135-project sample, not a prevalence estimate (Section 8.9).

- **Security of the controller host.** Treated as trusted infrastructure throughout. Scenario E′ is precisely what happens when that assumption fails, and the layered build does not survive it.

Chapter 11

Figures, Diagrams and Visual Documentation

Every figure in this report and in the manuscript is generated by a script from the repository and regenerates byte-identically. None was drawn by hand in a graphics editor. This chapter records how they are produced, the quality criteria they must satisfy, and the defects that the checking scripts were written to catch — including one that visual inspection cannot detect.

11.1 Generation pipeline

Figures come from two generators.

Data figures (Figures 3 and 5) are produced by `experiments/make_ablation_figures.py` using matplotlib 3.11.2, configured through `experiments/figstyle.py`. The style module fixes the palette, the typeface and `pdf.fonttype 42`, so TrueType fonts are embedded rather than rendered as Type 3, which several publishers reject.

Diagrams are produced by `docs/figures/make_diagrams.py`, which emits SVG directly rather than through a drawing library. The SVG is then converted to PDF and PNG with cairosvg. Writing the SVG directly is what makes the printed-size check of Section 11.3.1 possible: every text element's font size and the canvas width are both known quantities in the source.

Reproducibility. Re-running both generators in a clean environment reproduces every SVG byte-identically. The PDFs differ only in their embedded creation timestamps; the rendered content is identical.

11.2 The eight manuscript figures

No.	Title	Where in this report	What it is for
1	System architecture	Figure 4.1	The two-core partition and the three shared objects
2	Command execution flow	Figure 4.2	Where each check happens on the path from publisher to actuator
3	Command-path ablation results	Figure 8.1	Three properties across six designs; the independence result
4	Broker access control architecture	Figure 4.3	Per-client authorisation, and why the device identity cannot publish commands
5	Security compromise comparison	Figure 8.2	Nine cells indexed by which secret the adversary holds
6	Experimental setup	Figure 7.1	The simulation environment, with panel B marked as not yet measured
7	Compile-guard comparison	Figure 6.1	The defective and repaired build decisions side by side

No.	Title	Where in this report	What it is for
8	Transport security versus command authorisation	Figure 8.3	Scenarios E and E′ drawn together

A further seven diagrams — the UML command sequence, the deployment view, the methodology overview, the alarm state machine, the event sequence, the circuit wiring and the hardware measurement rig — exist as repository documentation and are not part of the manuscript. They are listed with their generators in Appendix H.

11.3 Quality criteria, and how each is enforced

Criterion	Enforced by	Failure mode it prevents
Vector format	generator output	Raster artefacts at print resolution
Fonts embedded, no Type 3	`pdf.fonttype 42`, `pdffonts` audit	Publisher rejection; substituted glyphs
≥ 6 pt printed text	`check_figure_type.py`	Labels illegible in print
No clipped labels	`check_figure_fit.py`	Text overflowing its box
Values match raw records	`validate_figures.py`	A figure drifting from the data
No baked-in cross-references	`validate_figures.py` §5	A renumbered table leaving the artwork wrong
Consistent colour semantics	review, documented in the caption	A colour meaning two different things

11.3.1 The printed-size defect

Every diagram in an earlier revision printed its smallest label at roughly 4 pt. This was invisible to inspection on screen, where the SVG renders at whatever size the viewer chooses, and it is not a property of the figure alone: it is a function of the canvas width and the scale fraction at which the figure is included.

The printed point size of a label is

$$pt_{\text{printed}} = font_units \times \frac{textwidth_{\text{pt}} \times width_fraction}{canvas_width_{\text{units}}}$$

with $textwidth_{\text{pt}} = 595.276 - 2 \times 72.27 = 450.74$ pt for the A4 one-column layout. A 17.5-unit label on a 1,900-unit canvas included at `\textwidth` therefore prints at 4.2 pt.

`experiments/check_figure_type.py` reads the `\includegraphics` lines from the manuscript, locates the matching SVG, computes the printed size of every text element and exits non-zero below a 6 pt floor. The repair was to reduce every canvas width — for example `security_paths` from its original width to 1,160 units — rather than to enlarge the type, because shrinking the canvas raises every label at once and preserves the layout.

11.3.2 The label-fit defect

A separate script, `experiments/check_figure_fit.py`, measures each label in the actual DejaVu Sans faces using Pillow, computes its extent from its anchor, and compares it

against the tightest enclosing rectangle and against the canvas. Forty-five overflowing labels were found and corrected on its first run. Measuring in the real font matters: an estimate from character counts misses the cases where it is wrong.

11.3.3 The baked-in cross-reference defect

Figure 4 carried the text "Measured in Table 3" inside its artwork. The credential table is Table 8. The reference was wrong, and more importantly it could not be corrected by LaTeX, because it was a drawn string rather than a `\ref`.

The text was removed, and `validate_figures.py` now fails the build if any diagram source contains a hard-coded `Table N` or `Figure N`. No figure may refer to another float by number.

11.3.4 Colour semantics

Two figures used red to mean both "a step blocked the attack" and "the attack succeeded", which inverts the reading between panels of the same document. The repair confined outcome colour to a single column in Figure 5 and recoloured the compile-time `#error` path green in Figure 7 — a build that refuses to compile is the desired outcome, not a failure.

Figure 8.1 panel (c) is the one deliberate inversion. There a high value is the failure, so the colour rule inverts with it and blue continues to mean "this design holds the property". The inversion is stated in the caption, because a reader should not have to infer it.

11.4 Verification of figure values against raw records

`experiments/validate_figures.py` recomputes every plotted value from the raw per-trial records in `experiments/results/ablation.jsonl`, and not from the derived summary file. Reading the summary would only confirm that the summary and the figure agree, which is the weaker of the two possible checks and would not detect an error introduced when the summary was produced.

The script runs 59 assertions across six sections and is part of the test suite. All pass at the current head.

11.5 Tables

Tables follow the same rule as figures: every count, proportion, interval and p-value in this report was recomputed from the raw artefacts during its preparation. The summary table at the end of Chapter 8 records each value against its recomputed figure so that a reader can see the check rather than take it on trust.

Table formatting uses `booktabs` rules only — no vertical rules and no double horizontal rules — and `longtable` wherever a table may cross a page boundary, which avoids the alternative of shrinking type below the legibility floor to force a fit.

11.6 Figure collection for submission

A separate distributable collection is built by `tools/build_figure_collection.py`: a 7.2 MB archive of 54 files, containing PDF, SVG and 600 DPI PNG renderings of each figure measured at `\textwidth`, with a manifest and SHA-256 checksums. `tools/build_figure_-preview.py` produces a 16-page preview document so that a reviewer can inspect all fifteen

figures without extracting the archive. Both are scripts rather than manual steps, so the collection cannot fall out of step with the figures it contains.

Chapter 12

Limitations and Threats to Validity

This chapter is written to be read before the conclusions rather than after them. Each limitation states what was not established, why, and what would be required to establish it. Nothing here is hedging language around a result claimed elsewhere; where a limitation contradicts a stronger reading of a result, the limitation is correct.

12.1 The central limitation: no hardware measurement

No physical ESP32 was available in the environment that produced these results. The probe that established this is recorded in Appendix I: there were no serial devices, no USB subsystem, no `arduino-cli`, no Python `serial` module, and the only network interfaces present were `eth0`, `ifb0` and `lo`.

Four consequences follow, and they are the four most significant limitations of this work.

No timing claim of any kind is supported. The host harness advances a logical clock. The safety latency is the same constant — 151 ms — in every trial, by construction. Latency variance in this environment is identically zero, which is not a finding about the firmware but a property of the simulator.

Logical isolation does not imply physical timing invariance. This distinction is maintained throughout the report and is restated here as a limitation rather than a nuance. The model establishes that the safety task's enabling condition is independent of every network-side variable, so no network behaviour can *block* it. The firmware's asymmetric mutex timeout makes the blocking component of the loop period zero by construction. Whether the *total* loop period is invariant to network load on real silicon is an empirical question about cache contention, bus arbitration, interrupt latency, radio duty cycle and thermal behaviour, and it has not been answered. Panel B of Figure 7.1 is labelled as not yet measured for this reason and that label must not be removed until data exists.

On-device memory safety is not established. The three million fuzzing iterations ran against a host build under AddressSanitizer and UndefinedBehaviorSanitizer. Neither sanitizer exists for the Xtensa target. Compiler and architecture differences — alignment requirements, integer width, stack layout — mean a host-clean parser is evidence for, but not proof of, an Xtensa-clean parser.

No radio-layer experiment was performed. The deauthentication detector's parameters were swept in simulation only. An RF-shielded enclosure is a hard prerequisite for a real deauthentication experiment, both for validity and for legality, and none was available.

The pre-registered analysis plan for conditions A–D — the Mann–Whitney U test, the sample sizes, the acceptance criteria — was committed *before any data existed* and is reproduced in Appendix I. Its purpose is that the test cannot be chosen after seeing the measurements. The runbook in `docs/HARDWARE_A_TO_C_RUNBOOK.md` specifies the procedure in enough detail to be executed by someone else.

12.2 Threats to internal validity

12.2.1 The harness is not the firmware

The firmware logic is compiled against stub headers in `tests/host/stubs/`. The logic itself is unmodified — the same `smart_home.ino` is compiled, and only the headers it includes differ — but the environment around it is a model. There is no radio, no true parallel execution, no contention for shared silicon and no real clock.

12.2.2 Four defects were found in the measurement infrastructure itself

This is reported as a threat to validity because two of the four produced results that flattered the intended conclusion, and because it bears on how much confidence the remaining results deserve.

1. **Unattributable availability.** The harness recorded that a command failed without recording why, so a refusal by policy and a refusal by rate limit were indistinguishable.
2. **A token-bucket refill introduced by the first repair.** The repair to the above changed the arrival pattern, altering the quantity being measured.
3. **A single-attempt measurement that measured bucket phase.** One attempt per trial measured where in the refill cycle the attempt landed rather than the admission rate. This is why the availability denominator is 1,250 rather than 300: multiple attempts per trial are required for the measurement to mean what it claims.
4. **A 40 ms button hold against a 40 ms debounce.** A boundary condition that made the legitimate local path unreliable in the harness and had nothing to do with the firmware.

A fifth defect, in the corpus rule, is described in Section 8.9: the rule returned zero against this project's *own* pre-repair guard, because it treated any mention of the authentication identifier as evidence that authentication was required, and in the defective guard that identifier appears negated.

Each repair is accompanied by a control that would fail if the defect recurred. That is the only reason the remaining results are reported with confidence: not because the harness is assumed correct, but because specific, named ways in which it was wrong are now tested against.

12.2.3 The vacuous invariant

For part of this project the formal specification passed every check while being structurally incapable of failing (Section 9.3). A passing formal check was taken as evidence for a property the check could not test. The repair, and the inverted configuration that must report a violation, are described in Chapter 9. The episode is a limitation of the original method, not of the tool.

12.2.4 No multiplicity correction

Twelve ablation configurations are compared and two Fisher tests are reported. No correction for multiple comparisons is applied, on the grounds that every arm is a fixed, pre-declared configuration rather than a selected subset. This is stated so that a reader can judge the decision rather than infer it. Both reported p-values are smaller than 10^{-100} and would

survive any correction, but that is a property of these particular results and not a general defence of the choice.

12.3 Threats to external validity

12.3.1 A single system

This is a case study of one firmware artefact on one microcontroller with one broker. The defects found are defects in this system. The *categories* of defect — an unverified configuration assumption, a vacuous specification, an unbounded drain — are argued to generalise; the specific instances are not claimed to.

12.3.2 The corpus is biased by construction

The 135-project corpus was assembled from 22 fixed GitHub queries with a minimum of three stars and an 80 MB size cap, excluding archived repositories and forks. The criteria and rejection counts are recorded in `corpus.json`.

GitHub projects with stars are not deployed products. The sample over-represents educational and hobby code and under-represents commercial firmware, which is largely not public. **The 18-of-135 figure is not a prevalence estimate and must not be read as one**: rule 2 over-counts monitoring projects by roughly half, as the manual reading of all 18 showed, and under-counts non-GPIO actuation by an unknown amount — three of the nine confirmed cases actuate through Modbus, RF or serial frames and were caught only by an incidental GPIO write.

Rule 1 returned **zero** hits. That is reported as a zero, and **no claim is made that the guard pattern is widespread**.

12.3.3 One broker, one version, one configuration

Experiment E6 ran against Mosquitto 2.0.18 over TLS 1.2 with a private certificate authority. Scenario E's containment depends on the access control list being deny-by-default and on the device identity lacking write access on the command topic. A different broker, a different version, or a permissive list would change the result. Other broker implementations — EMQX, HiveMQ, VerneMQ, AWS IoT Core — were not tested, and their authorisation models differ.

12.3.4 One framework, two versions

Resource cost was measured under arduino-esp32 2.0.17 and 3.3.12, which differ by roughly 175 kB of flash. Behaviour under the ESP-IDF directly, or under other framework versions, was not evaluated.

12.4 Threats to construct validity

Alarm activation is not security. In the `none`, `ratelimit` and `token` arms the alarm fires in all 300 trials, and all three are completely defeated by capture-and-replay. A single-metric evaluation would have concluded that no command defence performs as well as a message authentication code. Three metrics are reported for this reason.

Zero forgeries is not evidence of soundness. Three million fuzz iterations producing no accepted input is the expected outcome against a 128-bit tag whether the implementation is sound or not, since a chance forgery has probability 2^{-128}. The campaign is evidence of

memory safety and parser robustness, and is not independent evidence of authentication soundness.

Simulated latency measures the simulator. Stated above and repeated here because it is the most likely misreading of Chapter 8.

12.5 Scope exclusions

- **Side-channel and fault-injection attack.** Out of scope; no equipment. Power analysis against the HMAC computation is a known attack class against embedded cryptography and was not evaluated.
- **Flash encryption and secure boot.** Not enabled. Section 10.2 contains the flash-dump threat through authorisation rather than preventing extraction.
- **Over-the-air update security.** No update mechanism is implemented, so its security is neither claimed nor evaluated.
- **The controller host.** Treated as trusted infrastructure throughout. Scenario E′ is what happens when that assumption fails, and the layered build does not survive it.
- **Long-term operation.** No soak test. Flash wear from counter commits, clock drift, and memory fragmentation over months of operation were not evaluated.
- **Usability.** No human-factors study. The 41% command-refusal rate under the token bucket is reported as a measured proportion; no claim is made about how users would respond to it.

12.6 Known unresolved design problems

Two problems were identified by this work and are *not* fixed in the artefact. They are recorded here rather than in future work, because a reader of the code should know they are open.

Owner lockout under key compromise. Two configurations admit 0 of 1,250 operator commands, because the adversary advances the persisted replay counter beyond the operator's reach. A candidate repair is a counter-resynchronisation path authenticated by the local button, which is physically present and outside a remote adversary's reach. It has not been implemented or evaluated, and the obvious alternatives — a resynchronisation command over MQTT, or a counter window — weaken the replay resistance that the counter exists to provide.

Scenario E′. The layered configuration falls to a compromised controller host and survives a device flash dump, which is the opposite of the cost ordering a defender would choose. Mitigation requires an authority that does not reside on the controller — a hardware security module, a second factor for disarm, or a physically present confirmation — and none is implemented.

12.7 Items requiring confirmation by the author or supervisor

Recorded here so that no reader mistakes an unverified item for a verified one.

- The declaration, certificate and acknowledgement pages are unsigned drafts. The certificate page is explicitly marked as a template and carries no approval.

- **No supervisor approval has been obtained**, and no dissertation-overlap ruling has been made.
- **No institutional similarity screening report exists.** No similarity percentage is claimed anywhere in this report, official or otherwise.
- The competing-interest, funding and authorship-contribution declarations require explicit author confirmation before submission.
- No journal quartile is asserted anywhere in this work, because the ranking source could not be reached without circumventing bot protection, which was not attempted.
- Five references lack DOIs and require confirmation.

Chapter 13

Conclusion and Future Work

13.1 What the work set out to do, and what it found

This project began as the construction of a secure ESP32 intrusion alarm. It became a study of why the techniques normally relied upon to establish that such a system is secure did not establish it.

Four defects were found *in this project's own artefacts*. Each had survived the instrument that existed to catch it:

- A compile-time guard accepted transport security plus a broker login as evidence of publisher authority. It survived code review and emitted no diagnostic. The ablation found it, and measured its cost: alarm activation of 16.7% against 100% for the control, with the adversary disarming the device in 246 of 300 trials.
- A formal invariant stating that unauthenticated remote disarm never takes effect was *unfalsifiable by construction*. It passed continuous integration on every push. It was incapable of failing.
- The safety loop had no static bound on its own execution time. The defect had passed 9,600 adversarial trials, 3,000,000 fuzzing iterations, exhaustive model checking and 129 host tests. The response-time decomposition found it, because the decomposition required a bound that did not exist.
- Four further defects were found in the measurement infrastructure itself, two of which produced results that flattered the intended conclusion.

The pattern common to all of them is a single sentence, and it is the work's central claim:

> A security property asserted in build configuration, and a timing property asserted in design, are both unverified claims. Each can be falsified by a method that costs far less than the system it checks.

The guard asserted authorisation in a preprocessor expression. The specification asserted a refusal in a comment. The loop asserted a bound nowhere at all. In each case the assertion was accepted because it appeared in a location where assertions are not normally examined.

13.2 Objectives against outcomes

	Objective	Outcome
O1	Adequacy of core pinning with bounded queue discipline, and what can be settled without hardware	**Partially met.** Logical isolation proved exhaustively (660 states). Physical timing invariance not measured, and the boundary between the two is now stated precisely wherever the claim appears.
O2	Formally, whether a remote adversary can affect the safety function's control or data flow	**Met.** `SafetyCanAlwaysStep` holds in every reachable state under an unfair, unboundedly stalling, crashing network task.
O3	Which command-path defence preserves the safety function, availability and replay resistance	**Met.** The three properties fail independently. Only the counter-bound message authentication code holds all three; the rate limiter costs 41% of operator commands.
O4	Where broker authorisation and application-layer authentication fail, and whether either dominates	**Met, with a negative result.** Neither dominates. The access control list contains a device compromise the signature cannot; the signature contains a controller compromise the list cannot. The layered build falls to E′.
O5	A response-time bound, separating analytical from hardware terms	**Partially met.** Three of four interference terms removed by construction; the fourth requires hardware. The derivation exposed the unbounded drain.
O6	Whether the configuration defect generalises to comparable open-source projects	**Attempted; null result.** Rule 1: zero hits across 135 projects. Rule 2: nine confirmed cases, reported as an existence claim and not a prevalence estimate.
O7	A falsifiable measurement protocol for the unresolved timing claim, published whether or not it can be executed	**Met.** Pre-registered before any data existed, with the test, sample sizes and acceptance criteria fixed in advance (Appendix I).

Six of seven objectives are met or partially met. The two partial outcomes share one cause: the absence of physical hardware, which is the work's principal limitation (Chapter 12).

13.3 What this work establishes

1. **Security properties of a command path fail independently, and must be measured separately.** Alarm activation, operator availability and replay resistance were each preserved by a design that destroyed one of the others. A single-metric evaluation of these six designs would have reached the wrong conclusion, and the figure reporting them needs three panels for exactly that reason.

2. **Transport authentication is not publisher authorisation in MQTT 3.1.1.** The broker re-originates messages; the protocol carries no publisher identity to the subscriber. The cost of the unsound inference was measured rather than argued.

3. **Broker authorisation and application-layer authentication are complementary and neither subsumes the other.** Established against real broker software, because scenarios E and E′ carry the same valid signature and differ only in the transmitting identity — a distinction no amount of reasoning about the cryptography would have settled.

4. **A formal property must be shown to be falsifiable before its success is evidence.** The repair is not a better invariant; it is a configuration whose *required* outcome is a violation, with the specific invariant name required in the output. The same reasoning produced the negative control for trace conformance.

5. **Response-time analysis is a defect-finding instrument and not only a certification instrument.** Four other methods had passed the firmware; deriving the bound found the defect, because the derivation required a quantity the code did not have.

6. **Measurement infrastructure requires the same scrutiny as the artefact.** Four defects were found in the harnesses, two of them flattering. Every repair is now accompanied by a control that would fail if the defect recurred.

13.4 What this work does not establish

Restated compactly, because the conclusions should not be readable without it.

No timing measurement of any kind was taken. Physical timing invariance is unmeasured and unclaimed; logical isolation does not imply it. On-device memory safety is not established, the fuzzing having run on a host build under sanitizers that do not exist for the Xtensa target. No radio-layer experiment was performed. The corpus study makes no prevalence claim. The flash-dump containment depends on the access control list and on the device identity lacking command write access, and not on the key being protected — it is not. The layered configuration falls to a compromised controller host. Two design problems identified by this work remain open and unfixed: owner lockout under key compromise, and scenario E′.

13.5 Future work

13.5.1 Immediate: execute the pre-registered hardware protocol

Conditions A–D, as specified in Appendix I and `docs/HARDWARE_A_TO_C_RUNBOOK.md`. The analysis plan is already fixed, so the measurement can confirm or falsify the timing claim without the test being chosen after the data is seen. This is the single highest-value next step, because it is the only one that resolves a claim the work currently has to leave open. Conditions A–C require an ESP32 DevKit, a logic analyser or a second microcontroller as a timing reference, and a controlled network; condition D additionally requires an RF-shielded enclosure.

13.5.2 Repair the two open design problems

Owner lockout. A counter-resynchronisation path authenticated by the local button, which is physically present and outside a remote adversary's reach. The obvious alternatives

— a resynchronisation command over MQTT, or a tolerance window on the counter — weaken the replay resistance the counter exists to provide, so the design question is not trivial and the proposed repair needs evaluation against the same twelve-arm ablation before it can be claimed to work.

Scenario E′. Mitigation requires an authority that does not reside on the controller host: a hardware security module, a second factor for disarm, or a physically present confirmation for state-changing commands. Each shifts the cost of the attack rather than eliminating it, and the trade-off against operator availability — already shown to be a real cost — would have to be measured.

13.5.3 Extend the evaluation

Flash encryption and secure boot, which would address key extraction rather than containing it. Other broker implementations with different authorisation models — EMQX, HiveMQ, AWS IoT Core — since scenario E's containment is contingent on Mosquitto's deny-by-default list. On-device fuzzing, which requires an instrumentation approach that does not depend on host sanitizers. A soak test for flash wear from counter commits, clock drift and fragmentation over months.

13.5.4 Generalise the method rather than the finding

The corpus study attempted to generalise the *defect* and returned a null result. A more promising direction is to generalise the *instrument*: a static check for the class of unverified configuration assumption found here, validated against a labelled set of positive and negative controls rather than against a star-filtered GitHub sample. The eight controls built for this project's rule, including its own pre-repair guard, are the starting point for such a labelled set.

13.6 Closing remark

The most useful outcome of this project is not the firmware. It is the record of four methods that each reported success while a defect they were meant to catch was present, and of the specific, testable changes that make each method capable of failing. A test that cannot fail is not evidence, and that applies to a build guard, a model-checked invariant, a fuzzing campaign and a measurement harness alike.

References

Bibliography

[1] I. Behnke, L. Pirl, L. Thamsen, R. Danicki, A. Polze, and O. Kao, "Interrupting real-time IoT tasks: How bad can it be to connect your critical embedded system to the Internet?," in *Proc. IEEE 39th Int. Performance Computing and Communications Conf. (IPCCC)*, 2020. doi:10.1109/IPCCC50635.2020.9391536.

[2] I. Behnke, P. Wiesner, R. Danicki, and L. Thamsen, "A priority-aware multiqueue NIC design for real-time IoT devices," in *Proc. 37th ACM/SIGAPP Symp. on Applied Computing (SAC)*, 2022. doi:10.1145/3477314.3507165.

[3] I. Behnke, C. Blumschein, R. Danicki, P. Wiesner, L. Thamsen, and O. Kao, "Towards a real-time IoT: Approaches for incoming packet processing in cyber-physical systems," *Journal of Systems Architecture*, vol. 140, art. 102891, 2023. doi:10.1016/j.sysarc.2023.102891.

[4] F. Fischer *et al.*, "Analysis of industrial device architectures for real-time operations under denial of service attacks," arXiv:2007.08885, 2020.

[5] L. Meszlényi, J. Kahle, D. Püllen, S. Kowalewski, S. Katzenbeisser, and A. Kampmann, "Towards timing isolation for mixed-criticality communication in software-defined vehicles," arXiv:2508.13652, 2025.

[6] D. Hoornaert, G. Ghaemi, A. Bastoni, R. Mancuso, M. Caccamo, and G. Corradi, "Mcti: Mixed-criticality task-based isolation," *Real-Time Systems*, vol. 60, no. 2, pp. 328–365, 2024. doi:10.1007/s11241-024-09425-5.

[7] A. Burns and R. I. Davis, "Mixed criticality systems: A review," 14th edn., Dept. of Computer Science, University of York, Feb. 2026.

[8] S. Kremser and K. Graffi, "Evaluating the ESP32-S3 for Wi-Fi penetration testing through the development of Deauther32 and HackHeld32," *Sensors*, vol. 26, no. 11, art. 3287, 2026. doi:10.3390/s26113287.

[9] R. Cayre, D. Cauquil, and A. Francillon, "ESPwn32: Hacking with ESP32 system-on-chips," in *Proc. 17th IEEE Workshop on Offensive Technologies (WOOT)*, 2023.

[10] A. Carbajal and A. Jodeiri Akbarfam, "A software-defined testbed for quantifying deauthentication resilience in modern Wi-Fi networks," arXiv:2602.23513, 2026.

[11] I. Mudraje, K. Vogelgesang, J. Devreker, L. Gerhorst, P. Raffeck, P. Wägemann, and T. Herfet, "Reverse engineering the ESP32-C3 Wi-Fi drivers for static worst-case analysis of intermittently-powered systems," arXiv:2501.17684, 2025. doi:10.1145/3722572.3727926.

[12] M. A. Azhar, M. Azwar, and O. Asroni, "Performance and security analysis of MQTT over TLS on ESP32-based IoT systems," *J. Applied Informatics and Computing*, 2026.

[13] V. Ieropoulos *et al.*, "Empowering IoT security: On-device intrusion detection in resource constrained devices," arXiv:2605.13159, 2026.

[14] M. Krichen, "A survey on formal verification and validation techniques for Internet of Things," *Applied Sciences*, vol. 13, no. 14, art. 8122, 2023. doi:10.3390/app13148122.

[15] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[16] J. Yun, F. Rustamov, J. Kim, and Y. Shin, "Fuzzing of embedded systems: A survey," *ACM Computing Surveys*, vol. 55, no. 7, 2022. doi:10.1145/3538644.

[17] M. Bley, T. Scharnowski, S. Wörner, M. Schloegel, and T. Holz, "Protocol-aware firmware rehosting for effective fuzzing of embedded network stacks," in *Proc. 2025 ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2025. doi:10.1145/3719027.3765125.

[18] D. Tharindu, A. Jayasena, and P. Mishra, "SysFuSS: System-level firmware fuzzing with selective symbolic execution," arXiv:2602.02243, 2026.

[19] W. Alsabbagh, S. Amogbonjaye, C. Kim, and P. Langendörfer, "Pirates of the MQTT: Raiding IIoT systems with a rogue client," in *2024 8th Cyber Security in Networking Conference (CSNet)*, IEEE, 2024, doi:10.1109/CSNET64211.2024.10851733.

[20] H. Zeghida, M. Boulaiche, and R. Chikh, "Security of MQTT protocol: A brief overview," in *Proc. TACC 2024*, CEUR Workshop Proceedings, 2024.

[21] G. Amirkhanova, S. Ismailov, A. Amirkhanov, S. Adilzhanova, M. Zhasuzakova, and S. Chen, "A lightweight, end-to-end encrypted data pipeline for IIoT: An AES-GCM implementation for ESP32, MQTT, and Raspberry Pi," *Information*, vol. 17, no. 1, art. 33, 2026, doi:10.3390/info17010033.

[22] C. Yu, M. Qi, H. Luo, J. Tian, and J. Lu, "MQTT-L: A lightweight end-to-end secure protocol with renewable hash chains for IoT environments," *Journal of King Saud University Computer and Information Sciences*, vol. 38, no. 6, art. 539, 2026, doi:10.1007/s44443-026-00972-4.

[23] M. S. Wara and Q. Yu, "New replay attacks on ZigBee devices for Internet-of-Things (IoT) applications," University of New Hampshire; IEEE, 2020. Accession 10215772, NSF Public Access Repository, `par.nsf.gov/biblio/10215772`. *Venue unverified:* the accessible copy carries an IEEE copyright line but names no conference, and the publisher record could not be retrieved, so no venue, volume or page range is asserted here. The frame-counter denial-of-service mechanism relevant to Section 8.4 is stated in that paper's introduction as background, without a citation and without an experiment.

Appendix A

Repository Structure

The repository at commit 5fbe123 (5fbe1235a6633c6cef58a8e5e4c2a2c77c6b2ce4) — the commit this report was generated from. A document that records its own commit hash cannot record the commit that contains it, so the commit adding the rebuilt PDF is necessarily a child of the one named here; the sources it describes are identical in both.

Directory listing to two levels. Generated output, build artefacts and the model checker's trace files are omitted, so the listing describes the repository's source rather than the most recent build.

Directory	Files	Contents
(root)	10	Licence, citation metadata, top-level README, reproduction guide, changelog
.github/workflows	1	Continuous integration
docs	37	Analysis documents and the figure sources
docs/figures	1	The diagram generator
docs/images	26	Generated SVG, PDF and PNG figures
docs/media	2	
experiments	17	Harnesses, analysis scripts, figure generators, verification checks
experiments/broker	3	Real Mosquitto testbed: setup, ACL variants, scenario runner
experiments/corpus	8	Corpus builder, static rule, controls, commit pins, findings
experiments/hardware	4	Probe firmware, collection and pre-registered analysis (not yet run)
experiments/hardware/latency_probe	1	
experiments/results	12	Committed raw and aggregate experimental records
firmware/espnow_receiver	3	Companion receiver sketch (not part of the evaluation)
firmware/smart_home	6	The artefact under study: v2.2.0 sketch and its headers
formal	7	TLA+ specifications, three TLC configurations, runner, tla2tools.jar
paper	3	Manuscript source and build products
paper/submission	2	Highlights, declaration document, similarity-screening file
paper/submission/similarity	1	
report	16	This report: chapter sources, front matter, generated data
report/front	6	
report/generated	5	

Directory	Files	Contents
`tests/host`	2	Host test suite and the stub headers it compiles against
`tests/host/stubs`	9	
`tests/host/stubs/freertos`	4	
`tools`	7	Command signer, evaluator, figure and archive builders
`wokwi`	2	Simulator circuit definition

Appendix B

Source Code Inventory

Every source file in the repository with its language, total lines and non-comment, non-blank lines. **64 files, 7293 lines of code.** The firmware itself is a minority of the total; the verification and experimental infrastructure is the larger part, which is a deliberate consequence of treating that infrastructure as part of the artefact (Chapter 5).

The complete source is distributed as `FINAL_SUBMISSION_PACKAGE/source_code/COMPLETE_SOURCE_CODE_AND_EXPERIMENTS.zip`, which preserves the directory structure and contains no credentials, keys or environment-specific files. The repository's only key material is the published demonstration string in `secrets.example.h`, which the build explicitly refuses to accept as a real key.

File	Language	Lines	Code
`docs/figures/make_diagrams.py`	Python	1213	971
`experiments/ablation.cpp`	C++	632	384
`experiments/analyse.py`	Python	172	141
`experiments/analyse_ablation.py`	Python	377	333
`experiments/broker/run.sh`	Shell	32	14
`experiments/broker/setup.sh`	Shell	93	51
`experiments/broker/validate_acl.py`	Python	287	239
`experiments/campaign.cpp`	C++	289	209
`experiments/check_figure_fit.py`	Python	135	103
`experiments/check_figure_type.py`	Python	71	56
`experiments/corpus/analyse_guards.py`	Python	218	176
`experiments/corpus/build_corpus.py`	Python	136	119
`experiments/corpus/fetch_corpus.sh`	Shell	42	28
`experiments/corpus/test_rule.py`	Python	99	79
`experiments/figstyle.py`	Python	68	51
`experiments/fuzz_cmd.cpp`	C++	173	129
`experiments/hardware/analyse_hw.py`	Python	110	92
`experiments/hardware/collect.py`	Python	65	55
`experiments/hardware/latency_probe/latency_probe.ino`	Arduino/C++	53	31
`experiments/make_ablation_figures.py`	Python	182	138
`experiments/make_figures.py`	Python	91	74
`experiments/run.sh`	Shell	32	21
`experiments/run_ablation.sh`	Shell	50	35
`experiments/sensitivity.cpp`	C++	145	86
`experiments/test_conformance.py`	Python	165	132
`experiments/test_harness.py`	Python	174	141
`experiments/validate_figures.py`	Python	153	126
`firmware/espnow_receiver/alertmsg.h`	C/C++ header	78	57
`firmware/espnow_receiver/authmac.h`	C/C++ header	132	109
`firmware/espnow_receiver/espnow_receiver.ino`	Arduino/C++	80	55
`firmware/smart_home/alertmsg.h`	C/C++ header	78	57
`firmware/smart_home/authmac.h`	C/C++ header	132	109
`firmware/smart_home/secrets.example.h`	C/C++ header	43	15

File	Language	Lines	Code
firmware/smart_home/sensor_fusion.h	C/C++ header	58	37
firmware/smart_home/smart_home.ino	Arduino/C++	1166	836
firmware/smart_home/wifi_guard.h	C/C++ header	61	41
formal/AlarmFSM.cfg	TLC config	14	10
formal/AlarmFSM.tla	TLA+	146	124
formal/AlarmFSM_unsafe.cfg	TLC config	12	7
formal/Isolation.cfg	TLC config	14	10
formal/Isolation.tla	TLA+	143	124
formal/run_tlc.sh	Shell	48	34
tests/host/run_tests.sh	Shell	78	63
tests/host/stubs/Adafruit_GFX.h	C/C++ header	4	4
tests/host/stubs/Adafruit_SSD1306.h	C/C++ header	16	16
tests/host/stubs/Arduino.h	C/C++ header	61	61
tests/host/stubs/Preferences.h	C/C++ header	16	15
tests/host/stubs/PubSubClient.h	C/C++ header	33	33
tests/host/stubs/WiFi.h	C/C++ header	18	18
tests/host/stubs/WiFiClientSecure.h	C/C++ header	4	4
tests/host/stubs/Wire.h	C/C++ header	4	4
tests/host/stubs/esp_system.h	C/C++ header	7	7
tests/host/stubs/freertos/FreeRTOS.h	C/C++ header	29	29
tests/host/stubs/freertos/queue.h	C/C++ header	1	1
tests/host/stubs/freertos/semphr.h	C/C++ header	1	1
tests/host/stubs/freertos/task.h	C/C++ header	1	1
tests/host/test_firmware.cpp	C++	510	412
tools/build_figure_collection.py	Python	191	155
tools/build_figure_preview.py	Python	142	125
tools/build_report_data.py	Python	377	315
tools/build_source_archive.py	Python	195	153
tools/eval_commands.py	Python	125	91
tools/measure.py	Python	122	99
tools/sign_command.py	Python	61	47
64 files		**9458**	**7293**

Appendix C

Firmware Configuration Reference

C.1 Build-time switches

Macro	Default	Effect
`AUTH_COMMANDS`	0	Requires an HMAC-SHA256 tag and a strictly increasing counter on every remote command. **A deployment should set this to 1.**
`ALLOW_REMOTE_DISARM`	0	Permits remote disarm. Combined with `AUTH_COMMANDS 0` it is rejected at compile time unless the acknowledgement macro below is defined.
`ACKNOWLEDGE_-UNAUTHENTICATED_DISARM`	undefined	The deliberate escape hatch. Named for what the builder is accepting. Exists only so the research baseline stays reproducible.
`MQTT_USE_TLS`	—	TLS transport to the broker.
`SECRET_CMD_KEY`	demo	A public demonstration string. A build enabling `AUTH_COMMANDS` without overriding it is rejected at compile time, because authentication against a published key is not authentication.

C.2 Compile-time constants

Constant	Value	Why this value
`CMD_Q_LEN`	4	Command queue depth. Two `static_asserts` tie it to the drain bound.
`CMD_DRAIN_PER_PASS`	4	Equal to `CMD_Q_LEN`, which is what makes the loop's execution time statically bounded (Section 6.7.2).
Event queue depth	16	Drop-oldest. The queue never blocks the safety task.
`AUTH_TAG_BYTES`	16	128-bit truncated HMAC-SHA256 tag.
`DEAUTH_THRESHOLD`	12	Raised from 8 after the sensitivity sweep showed the original value firing in 79.9% of benign runs.
Detection window	2000 ms	Held fixed while the threshold was swept.
Exit / entry delay	10 s	
Alarm duration	30 s	

C.3 Command wire format

```
VERB:CTR:hex(HMAC-SHA256(key, "VERB:CTR")[0:16])
```

Listing C.1: The authenticated command format. The counter is committed to non-volatile storage *before* the command is acted upon.

Committing the counter before acting is deliberate: a reset between acting and committing would otherwise leave the counter below a value already used, reopening the replay window.

Appendix D

Formal Specification Listings

The complete text of `formal/AlarmFSM.tla` and `formal/Isolation.tla` is in the repository and in the source archive. This appendix records the configurations and their required outcomes; Chapter 9 discusses them.

Configuration	Constants	Result	Required
`AlarmFSM.cfg`	`MaxCtr = 4`, `AllowUnauthDisarm = FALSE`	166 generated, 25 distinct, no error	pass
`Isolation.cfg`	`QCap = 16`, `MaxEvents = 8`	2,686 generated, 660 distinct, no error	pass
`AlarmFSM_unsafe.cfg`	`MaxCtr = 4`, `AllowUnauthDisarm = TRUE`	`NoUnauthRemoteDisarm` violated	**violation**; a pass fails the build

All three are re-run by `formal/run_tlc.sh` on every push. The third requires the specific string `Invariant NoUnauthRemoteDisarm is violated` in TLC's output, not merely a non-zero exit, so that a syntax error or a missing constant cannot be mistaken for the expected violation.

Appendix E

Test Suite Reference

Suite	Scale	What it would catch
`tests/host/run_tests.sh`	129 assertions	Firmware logic defects. Includes three *negative build* tests that require a defective configuration to fail to compile; a successful build is a test failure.
`formal/run_tlc.sh`	3 configurations	A vacuous invariant recurring silently.
`experiments/test_conformance.py`	19,529 transitions	The firmware diverging from the specification. Includes a negative control that must be rejected.
`experiments/test_harness.py`	metamorphic and differential properties	Harness defects: determinism, seed sensitivity, monotone attack effort, mode separation, and agreement of the HMAC payload format with an independent implementation.
`experiments/corpus/test_rule.py`	8 controls	A static rule whose polarity is wrong. Includes this project's own pre-repair guard as a positive control and its repaired guard as a negative control.
`experiments/validate_figures.py`	59 assertions	A figure drifting from the raw records, or a baked-in cross-reference.
`experiments/check_figure_type.py`	all figures	A label printing below 6 pt.
`experiments/check_figure_fit.py`	all labels	A clipped label, measured in the real font.

All suites pass at commit 5fbe123.

Appendix F

Build and Deployment Instructions

F.1 Prerequisites

`arduino-cli` with the `esp32:esp32` core; the `Adafruit SSD1306`, `Adafruit GFX` and `PubSubClient` libraries; GCC 13 or later for the host tests; Python 3.11 or later; Java for `tla2tools.jar`; Mosquitto 2.0.x and OpenSSL for the broker experiment; and TEX Live for the documents.

F.2 Firmware

```
cp firmware/smart_home/secrets.example.h firmware/smart_home/secrets.h
# Edit secrets.h: set Wi-Fi credentials and REPLACE the demonstration keys.
# The build refuses AUTH_COMMANDS=1 with the published demonstration key.

arduino-cli compile --fqbn esp32:esp32:esp32 firmware/smart_home \
  --build-property "compiler.cpp.extra_flags=-DAUTH_COMMANDS=1"
arduino-cli upload  --fqbn esp32:esp32:esp32 -p /dev/ttyUSB0 firmware/smart_home
```

F.3 Verification and experiments

```
bash tests/host/run_tests.sh          # 129 assertions, incl. negative build tests
bash formal/run_tlc.sh                # 3 configurations; the third must report a violation
bash experiments/run.sh               # E1, E2: the adversarial campaign
bash experiments/run_ablation.sh      # E3: 12 configurations, 3,600 trials
bash experiments/broker/setup.sh      # E6: CA, certificates, password and ACL files
bash experiments/broker/run.sh        # E6: 8 scenarios against Mosquitto
python3 experiments/validate_figures.py
```

F.4 Documents

```
cd paper  && pdflatex manuscript.tex   # three passes
cd report && pdflatex report.tex       # three passes, for the ToC, LoF and LoT
```

F.5 Deployment guidance

A deployment should set `AUTH_COMMANDS 1` with a key that is not the published default, use per-client broker credentials with a deny-by-default access control list that does *not* grant the device identity write access on the command topic, and enable flash encryption and secure boot — which this firmware does not. The two open design problems of Section 12 apply to any such deployment and are not fixed.

Appendix G

Experiment Catalogue

Every experiment with its evidence class, artefact and scale. The evidence class is the distinction maintained throughout this report: a formal result and a simulated result support different claims, and the hardware conditions support none because they produced no data.

ID	Experiment	Evidence class	Artefact	Scale
E1	Adversarial campaign, baseline build	Simulation	`summary.json`	3,000 trials
E2	Adversarial campaign, hardened build	Simulation	`summary.json`	3,000 trials
E3	Command-path ablation	Simulation	`ablation.jsonl`	3,600 trials, 12 configurations
E4	Host-build fuzzing under ASan/UBSan	Simulation	`fuzz.json`	3,000,000 iterations
E5	Detector parameter sensitivity sweep	Simulation	`sensitivity.jsonl`	180 grid cells
E6	Access control against a live broker	Testbed measurement	`broker_acl.jsonl`	8 scenarios
E7	Open-source corpus study	Corpus study	`corpus/corpus.json`	135 projects
E8	Trace conformance against the TLA+ relation	Formal + simulation	`test_conformance.py`	19,529 transitions
E9	Model checking, three configurations	Formal	`formal/`	25 and 660 distinct states
--	Hardware conditions A–D	*not executed*	--	*no data exists*

Appendix H

Figure Index

Every figure with its title, file and the script that generates it. No figure was drawn by hand.

No.	Title	File	Generator
1	System architecture	`architecture.pdf`	`docs/figures/make_diagrams.py`
2	Command execution flow	`command_flow.pdf`	`docs/figures/make_diagrams.py`
3	Command-path ablation results	`ablation_results.pdf`	`experiments/make_ablation_figures.py`
4	Broker access control architecture	`acl_architecture.pdf`	`docs/figures/make_diagrams.py`
5	Security compromise comparison	`credential_matrix.pdf`	`experiments/make_ablation_figures.py`
6	Experimental setup	`experimental_setup.pdf`	`docs/figures/make_diagrams.py`
7	Compile-guard comparison	`guard_decision.pdf`	`docs/figures/make_diagrams.py`
8	Transport security versus command authorisation	`security_paths.pdf`	`docs/figures/make_diagrams.py`
Repository documentation figures, not part of the manuscript			
—	Command sequence (UML)	`command_sequence.pdf`	`docs/figures/make_diagrams.py`
—	Deployment view	`deployment.pdf`	`docs/figures/make_diagrams.py`
—	Methodology overview	`methodology.pdf`	`docs/figures/make_diagrams.py`
—	Alarm state machine	`state_machine.pdf`	`docs/figures/make_diagrams.py`
—	Event sequence	`event_sequence.pdf`	`docs/figures/make_diagrams.py`
—	Circuit wiring	`circuit_wiring.pdf`	`docs/figures/make_diagrams.py`
—	Hardware measurement rig (not built)	`hardware_rig.pdf`	`docs/figures/make_diagrams.py`

Appendix I

Hardware Validation Protocol

This protocol was not executed. No hardware data exists. It is published so that the timing claim left open by this work is falsifiable by someone else, and so that the analysis cannot be chosen after the data is seen.

I.1 The probe that established hardware was unavailable

Recorded so that the absence is a finding rather than an assertion. In the environment that produced the committed results there were no `/dev/ttyUSB*` or `/dev/ttyACM*` devices, no USB subsystem, no `arduino-cli` on the path, no Python `serial` module, and the only network interfaces present were `eth0`, `ifb0` and `lo` — no wireless interface of any kind. An RF-shielded enclosure, a hard prerequisite for condition D, was likewise unavailable.

I.2 The rig

A device under test and a separate probe microcontroller. The device under test raises a GPIO pin on sensor confirmation and again on siren assertion; the probe watches both and timestamps them with its own hardware timer, so the measurement does not depend on the device under test's own clock or on its serial output.

The two boards must share a common ground. Without it they have no shared reference and the readings are meaningless. This is the most common way this rig is got wrong.

I.3 The four conditions

Cond.	Setup	Shielding	What it would establish
A — idle	Wi-Fi connected, broker reachable, no other traffic	none	The control distribution and the probe's own noise floor. Everything else is compared against this.
B — legitimate load	Frequent status publication, large retained payloads	none	Whether ordinary operation perturbs the safety path. **A difference here would be the most damaging result for the design**, because it needs no adversary.
C — command stress	High-rate publication on the device's own command topic	none	The hardware counterpart of the simulated flood: where the isolation claim is actually tested.
D — radio stress	802.11 deauthentication and disassociation frames	**required**	The central open question: whether the mitigation holds under load on the radio itself rather than on the IP path.

The device must not be reflashed between conditions. The firmware must be byte-identical across all four, or the comparison is between two systems rather than two load levels.

I.4 The pre-registered analysis

Committed before any data existed, and **not to be substituted after seeing the data**.

Two-sided Mann–Whitney U test with tie correction, each condition against condition A. $\alpha = 0.05$, power 0.80, equal n per group. The planned sample size is $n = 200$ **per condition**, which detects a shift of 0.29 standard deviations — 29 µs at an idle spread of 100 µs, 52 µs at 180 µs, 87 µs at 300 µs. The effect size is expressed in standard deviations because the idle spread is unknown until condition A is run.

n per cond.	$A = P(X > Y)$	Shift (SD)	sd = 100 µs	sd = 180 µs	sd = 300 µs
30	0.709	0.78	78	140	233
50	0.662	0.59	59	106	177
100	0.614	0.41	41	74	123
200	**0.581**	**0.29**	**29**	**52**	**87**
400	0.557	0.20	20	37	61

A non-parametric test is pre-registered rather than a t-test because interrupt-driven latency distributions are routinely skewed with a long upper tail, and because the 95th percentile and maximum matter more for a safety function than the mean.

I.5 Status of the tooling

Component	**Status**
`experiments/hardware/latency_probe/`	Probe firmware, written, not flashed
`experiments/hardware/collect.py`	Serial collection, written, appends across conditions
`experiments/hardware/analyse_hw.py`	Summary statistics, the pre-registered Mann–Whitney U, and the box plot. **Verified end to end on explicitly synthetic input**, labelled as a script self-test. The synthetic file was deliberately not committed, so that no reader can mistake it for data.
`results_hw_TEMPLATE.csv`	Schema only: `condition,index,latency_us`. **Contains no rows.**
`docs/HARDWARE_A_TO_C_RUNBOOK.md`	Step-by-step procedure for A–C, executable by someone else

Conditions A–C require an ESP32 DevKit, a logic analyser or a second microcontroller as a timing reference, and a controlled network. Condition D additionally requires an RF-shielded enclosure and the appropriate permission; the repository publishes no attack tooling for it.

Appendix J

Journal Submission Status

Recorded as status rather than as achievement. The manuscript has **not** been submitted to any journal.

Requirement	Status	Evidence or action needed
Guide for Authors obtained	verified	Retrieved from the publisher's own site
Page and reference limits	verified	31 pages, 22 references, 0 dangling
Abstract length	verified	249 words, within the stated limit
Keywords	verified	7, within the stated limit
Structured declarations	verified	Present in the publisher's named sections
Highlights	verified	5 bullets, longest 80 of 85 characters
Figure format and font embedding	verified	All 8 fully embedded, 0 Type 3
Journal quartile and metric years	**unverified**	The ranking source returned an automated challenge. **Not circumvented**; no quartile is asserted anywhere in this work.
Competing-interest declaration	**author confirmation required**	Drafted; must not be declared on the authors' behalf
Funding declaration	**author confirmation required**	Drafted; no funding is invented
Authorship contribution statement	**author confirmation required**	The sole-author versus supervisor-co-author question is the author's to settle
Corresponding author's email and postal address	**required**	Not present on the title page
Institutional similarity report	**outstanding**	No licensed tool was available. **No similarity percentage is claimed, official or otherwise.**
Supervisor approval	**outstanding**	Not obtained
Dissertation-overlap ruling	**outstanding**	Not made
Five missing DOIs	**outstanding**	`mcreview2026`, `espwn2023`, `mqtttls2026`, `zeghida2024`, and an ISBN for `lamport2002`

Appendix K

Reproducibility Record

K.1 What was re-verified during the preparation of this report

- **Every reported count against its raw artefact.** 6,000 campaign trials; 3,600 ablation trials (9,600 adversarial trials in total); 3,000,000 fuzz iterations; 180 sensitivity cells; 8 broker scenarios; 135 corpus projects; 19,529 trace transitions.
- **Both Fisher p-values recomputed** from the raw records: 3.53×10^{-184} and 9.83×10^{-119}.
- **The Wilson interval recomputed:** [12.9, 21.3].
- **All three TLC configurations re-run:** 25 and 660 distinct states, and the required violation in the third.
- **The broker experiment re-run** against a freshly installed Mosquitto 2.0.18 in a clean environment. All eight verdicts identical; only log timestamps and random client identifiers differed.
- **Every figure regenerated.** All SVGs byte-identical; the PDFs differ only in embedded creation timestamps.
- **The full test suite:** 129 host assertions, 8 corpus controls, the harness metamorphic and differential properties, and 59 figure assertions. All pass.

K.2 Determinism

Seeds are fixed at 1000, 2000 and 7. The ablation harness forks per trial, so one trial cannot influence the next through accumulated state. All statistics are implemented with the Python standard library only, so no external package version can alter a reported value.

K.3 What cannot be reproduced

The hardware conditions, because they were never run. The corpus study depends on the state of third-party repositories at fetch time and on the GitHub search API, so a re-fetch will not return the same 135 projects; the fetched corpus and the rejection counts are committed for this reason, and the eight rule controls are reproducible independently of the corpus.